



# Machine Learning

CMPE 257

Support Vector Machines &  
Regularization Methods

Instructor: Zara Hajihashemi, Ph.D.

# Mid Term Exam

## Format

- Duration 180 minutes
- Multi-choice questions
- Short Answers
- Computational Questions

## Rules

- No laptop or Electronic devices
- Simple Calculator is needed
- No Brainstorming with LLMs
- Mid term is 25% of Final Grade
- One page cheatsheet (front & back)

# Agenda

- Hyperplanes
- Maximal margin classifier
- Support vector classifier
- Support vector machine
- Regularization Methods

# Support Vector Machine

## Introduction

Support vector machine (SVM) is a supervised method for binary classification (two class). It is a generalization of 1 and 2 below.

1. **Maximal margin classifier:** only applicable to linearly separable data.
2. **Support vector classifier:** can be applied to data that is not linearly separable.  
Decision boundary still linear.
3. **Support vector machine:** non-linear decision boundary.

# Hyper Plains

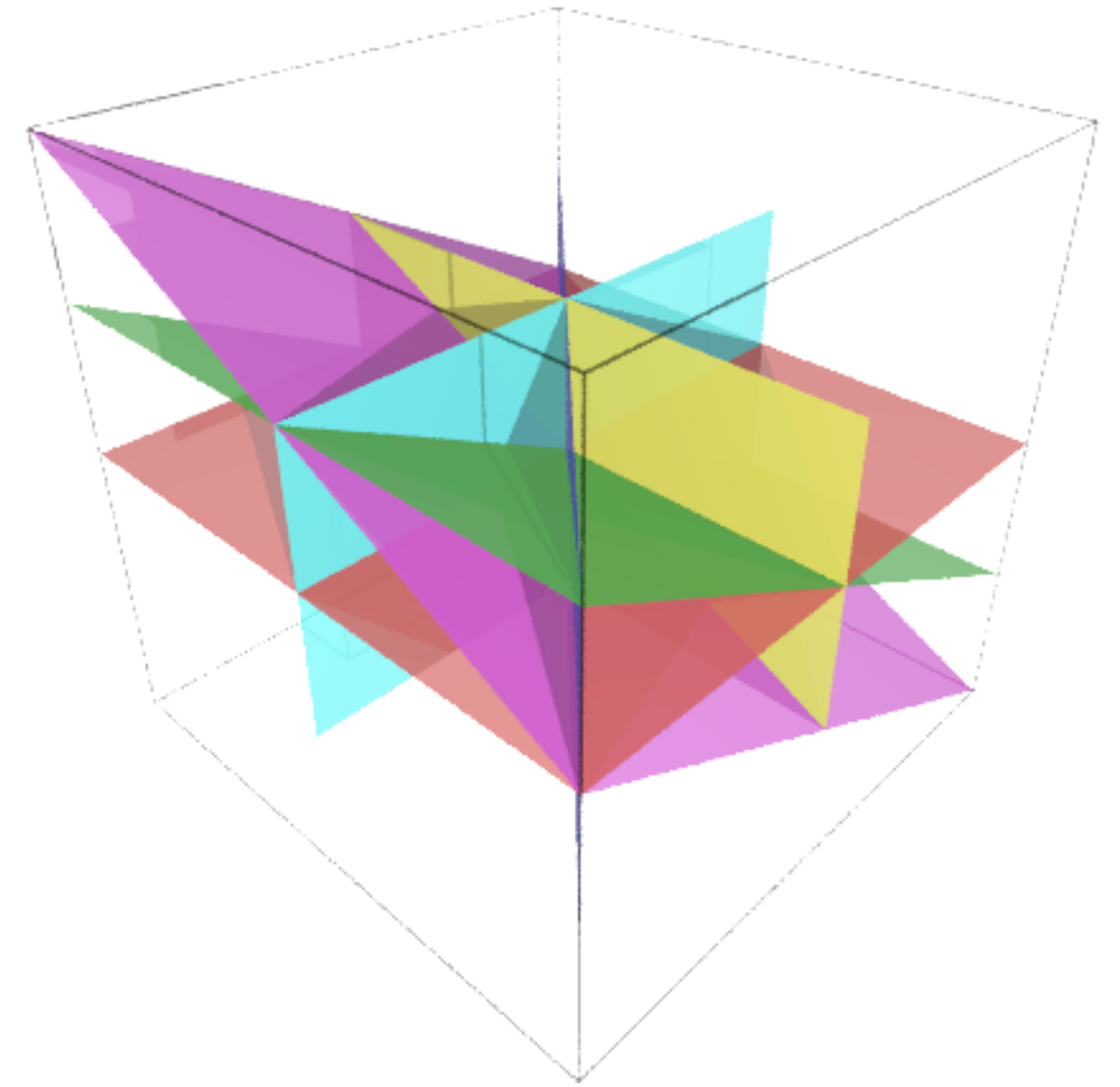
# What is a hyperplane

## Formal Definition

In  $p$ -dimensional space, a hyperplane is a  $(p-1)$ -dimensional affine subspace.

In 2D, a hyperplane is a flat 1D subspace, aka a line.

In 3D, a hyperplane is a flat 2D subspace, aka a plane.



# Hyperplanes

## Mathematical Definition

A 2D hyperplane is defined by the equation

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 = 0$$

By “define”, we mean that any  $\mathcal{X} = (x_1, x_2)$  for which the above equation holds is a point on the hyperplane.

Note that the above is the equation of a line, aka a hyperplane in 2D.

.

# Hyperplanes

## Mathematical Definition

A  $p$  dimensions, hyperplane is defined by the equation

$$\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p = 0$$

Similarly any  $\mathcal{X} = (x_1, x_2, \dots, x_p)$  for which the above equation holds is a point on the hyperplane.

# Separating Hyperplanes

## Mathematical Definition

Instead of a point on the hyperplane, consider  $X$  for which

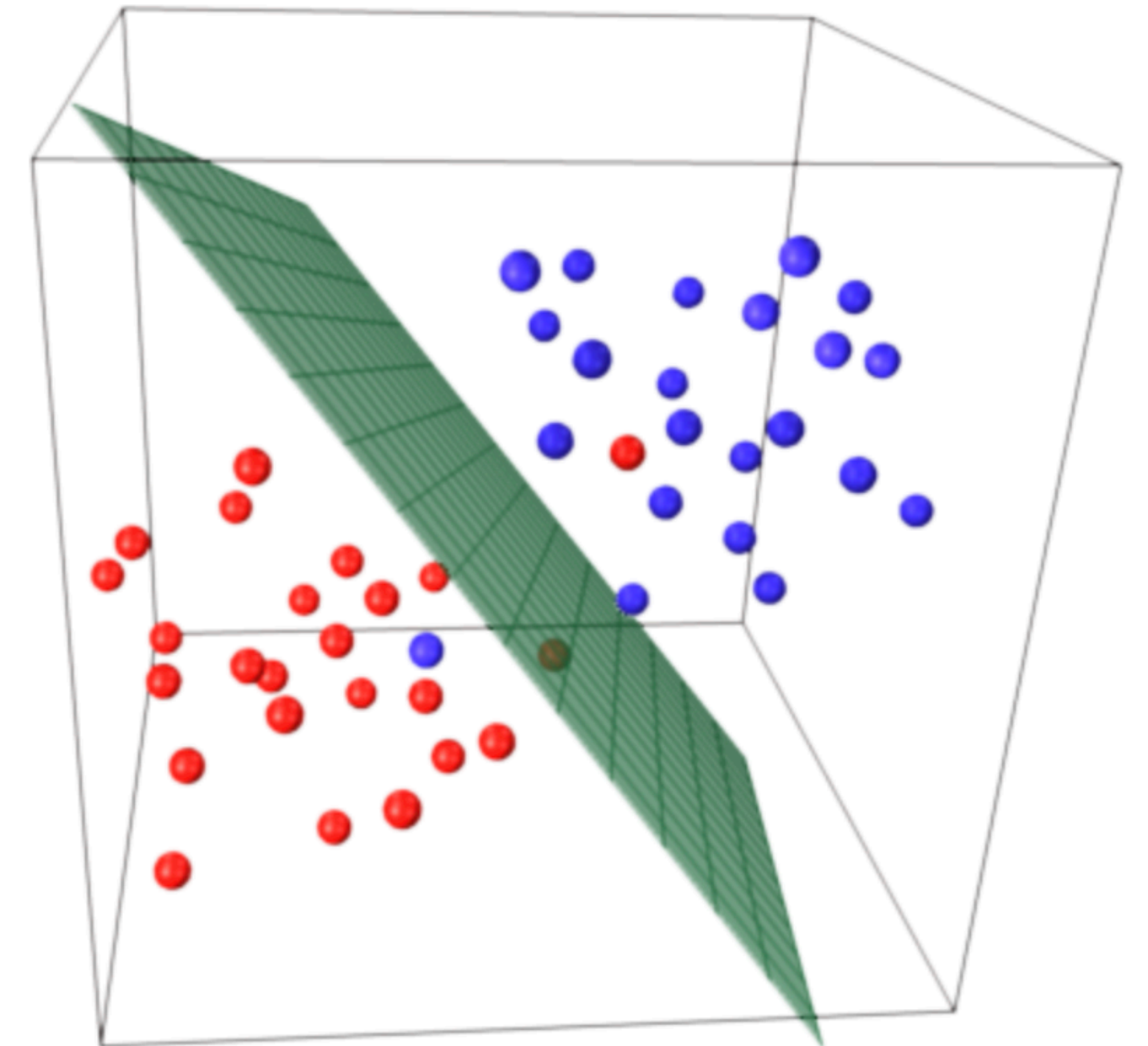
$$\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p > 0$$

This point lies on one side of the hyperplane.

An  $X$  for which  $\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p < 0$

lies on the other side of the hyperplane.

So in this way, We can think of the hyperplane as dividing the  $p$ -dimensional space into two halves.



# Separating Hyperplanes

This hyperplane in 2 dimensions is the line

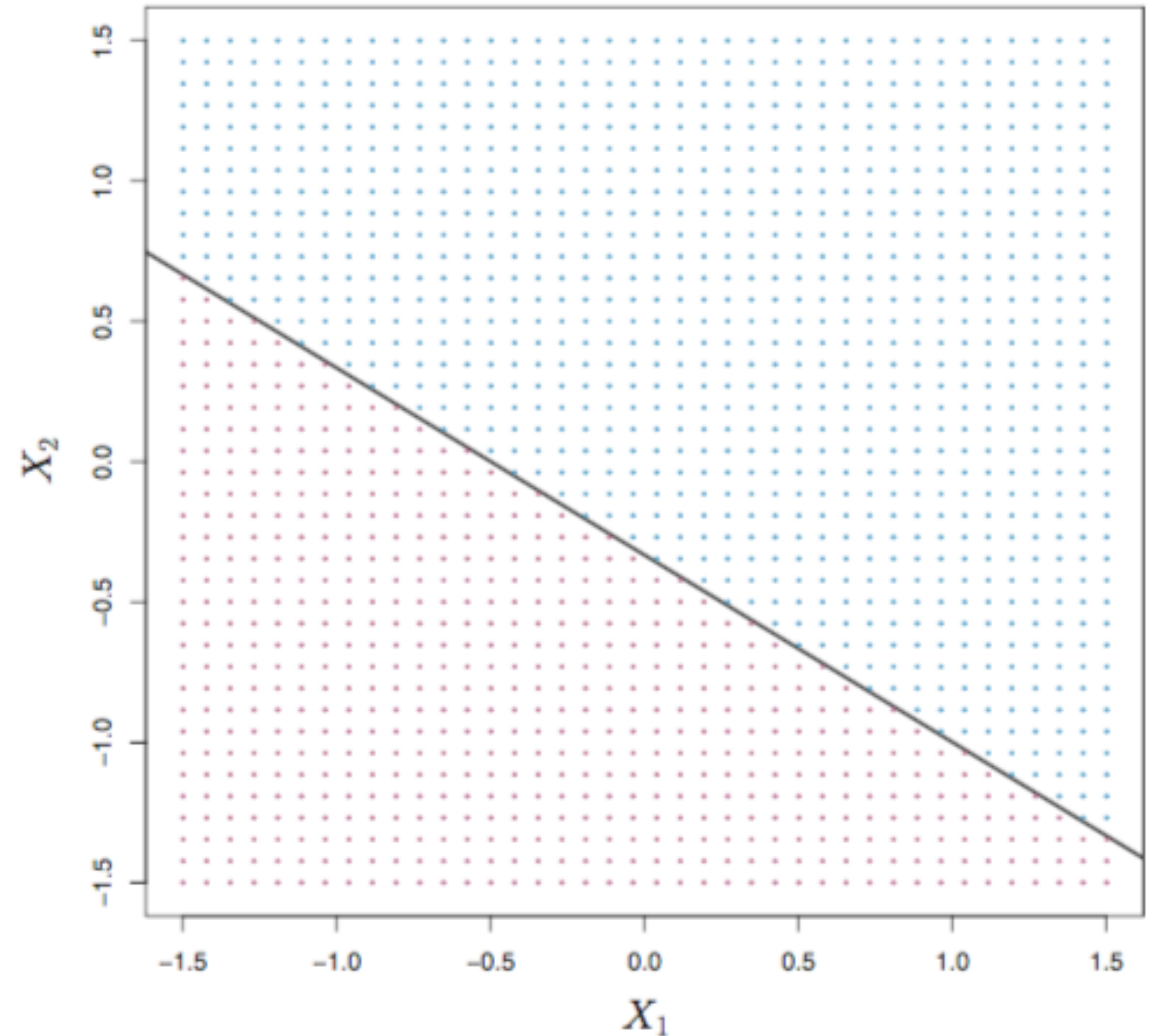
$$1 + 2X_1 + 3X_2 = 0$$

The blue region is the set of points for which

$$1 + 2X_1 + 3X_2 > 0$$

The purple region is the set of points for which

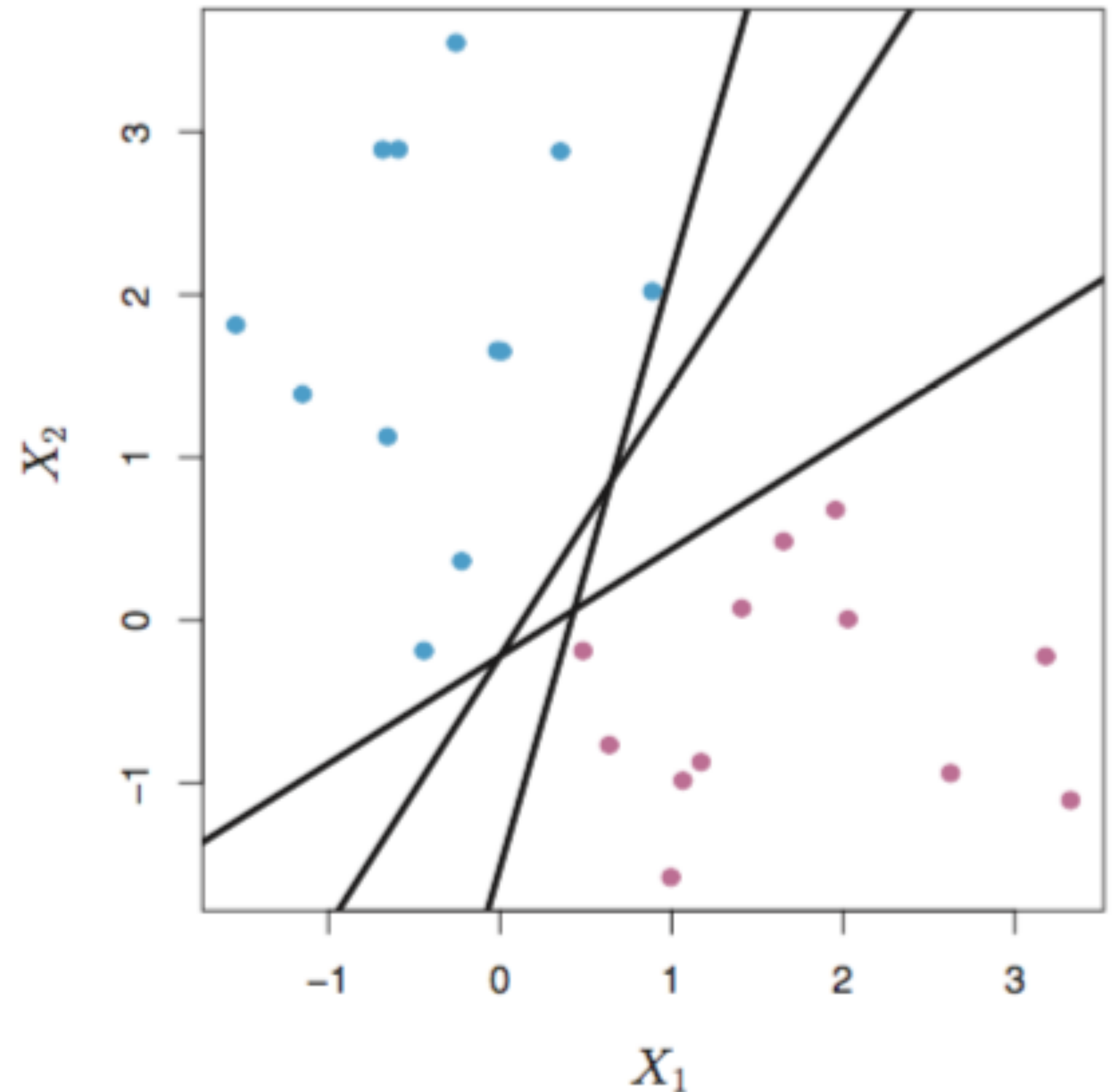
$$1 + 2X_1 + 3X_2 < 0$$



# Separating Hyperplanes Classifier

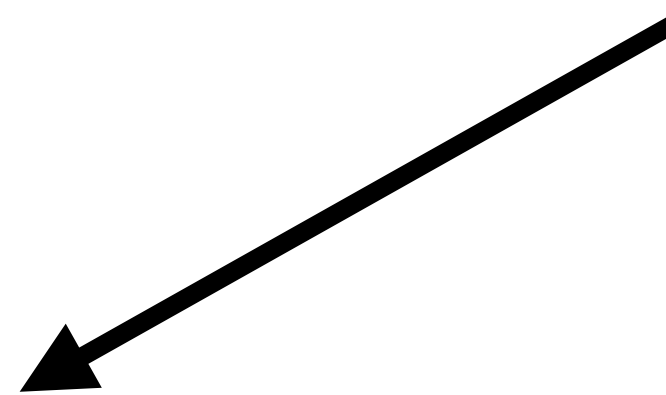
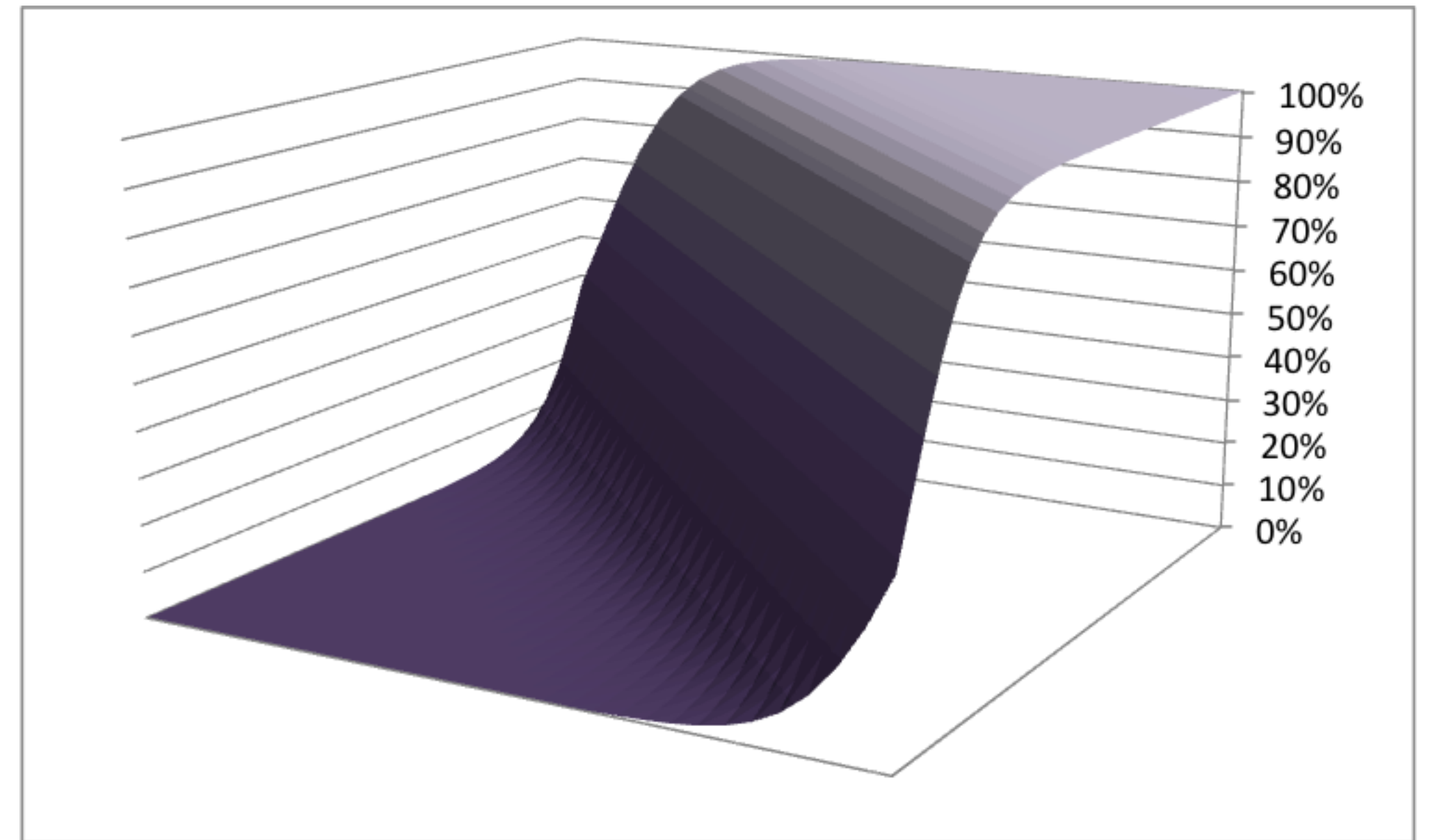
**Idea:** Use a separating hyperplane for binary classification.

**Key assumption:** Classes can be separated by a linear decision boundary.



# Separating Hyperplanes Classifier

**Note:** Logistic regression can also be thought of as finding a **separating hyperplane**.



**Logistic regression** finds a linear boundary by modeling probabilities.

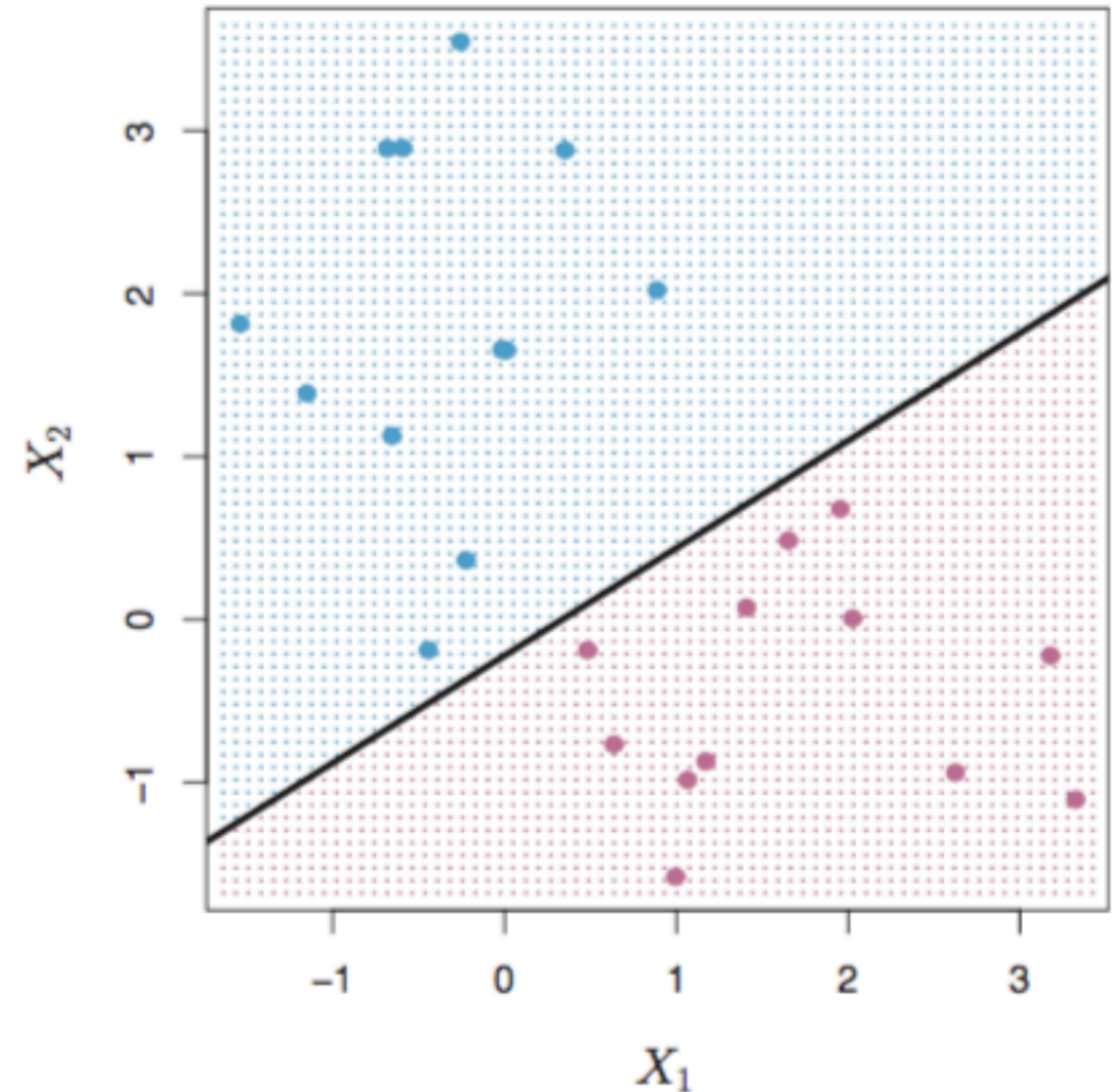
**SVMs** (and maximal margin classifiers) look for the boundary that maximizes the margin (the distance between the classes and the hyperplane).

# Separating Hyperplanes Classifier

**To classify new data points:** Assign class by location of new data point with respect to the hyperplane.

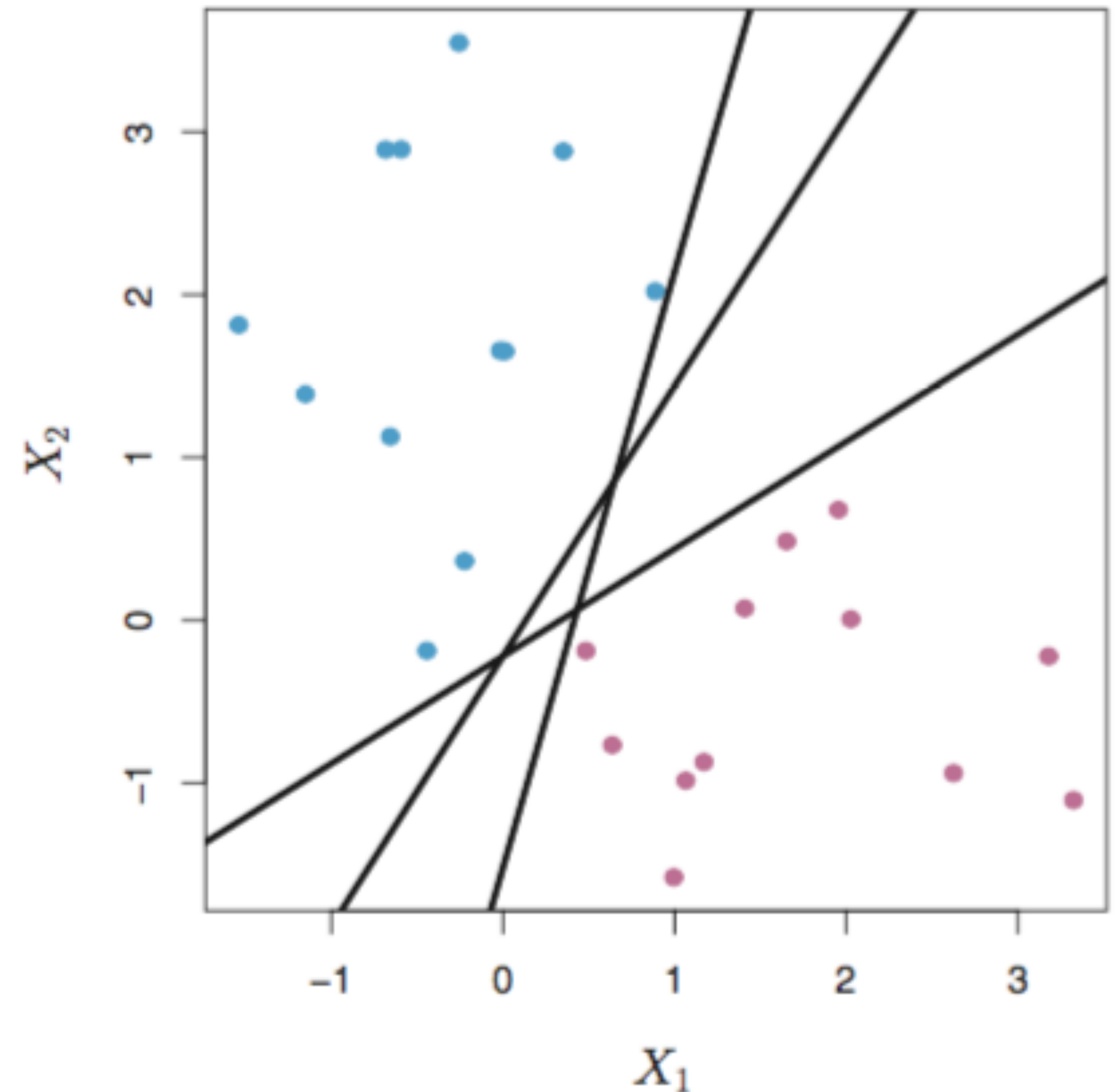
$$\hat{y} = \text{sign}(\beta_0 + \beta_1 x_1 + \dots + \beta_p x_p)$$

The **farther away** a point is from the separating hyperplane, the more confident we are about its class assignment.



# Separating Hyperplanes Classifier

**Notice** that for a linearly separable dataset, there are many possible separating hyperplanes that divide the dataset into two classes (in fact, an infinite number).

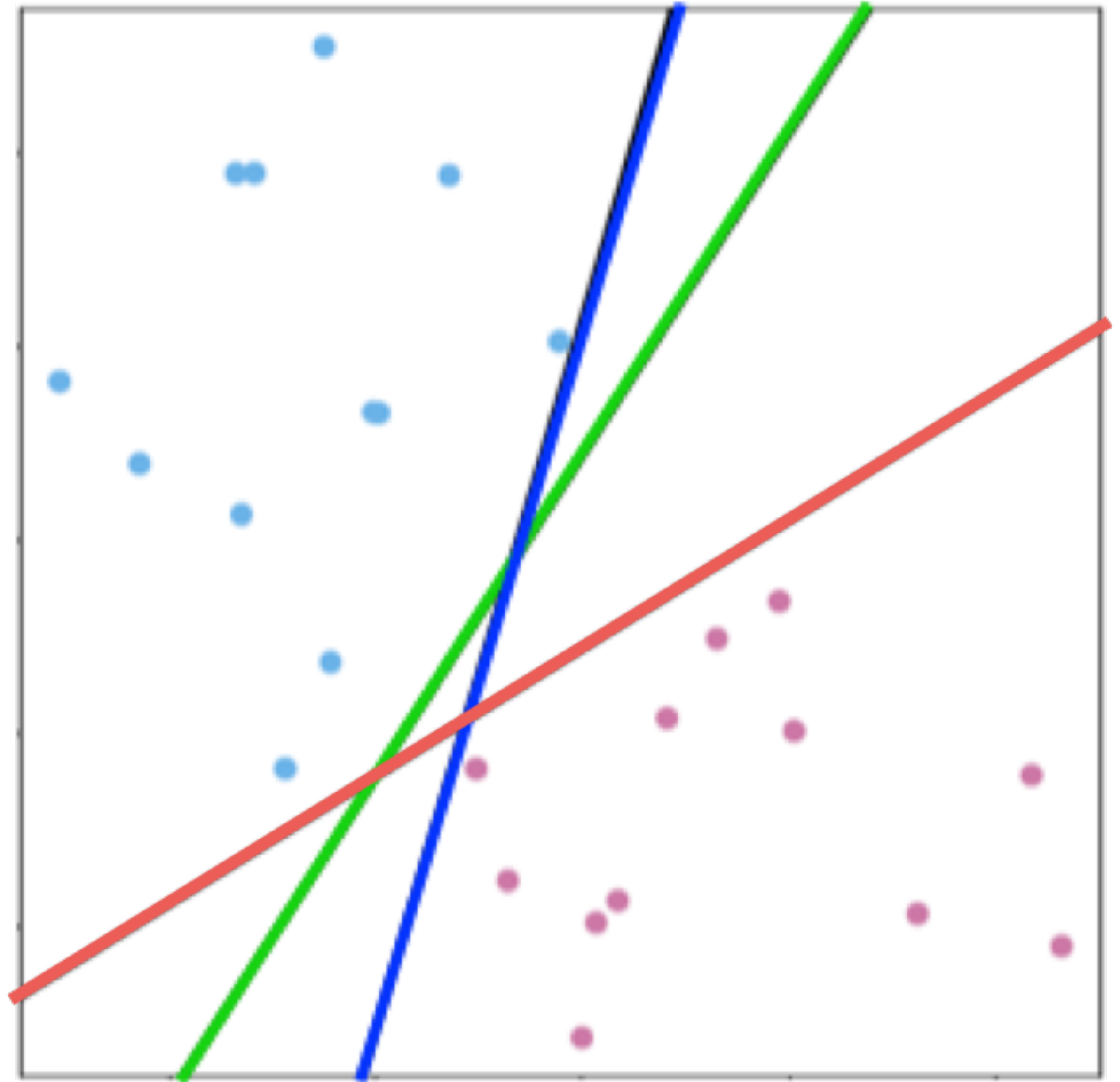


# Maximal Margin Classifier

# Which Decision Boundary?

*which decision boundary should we choose?*

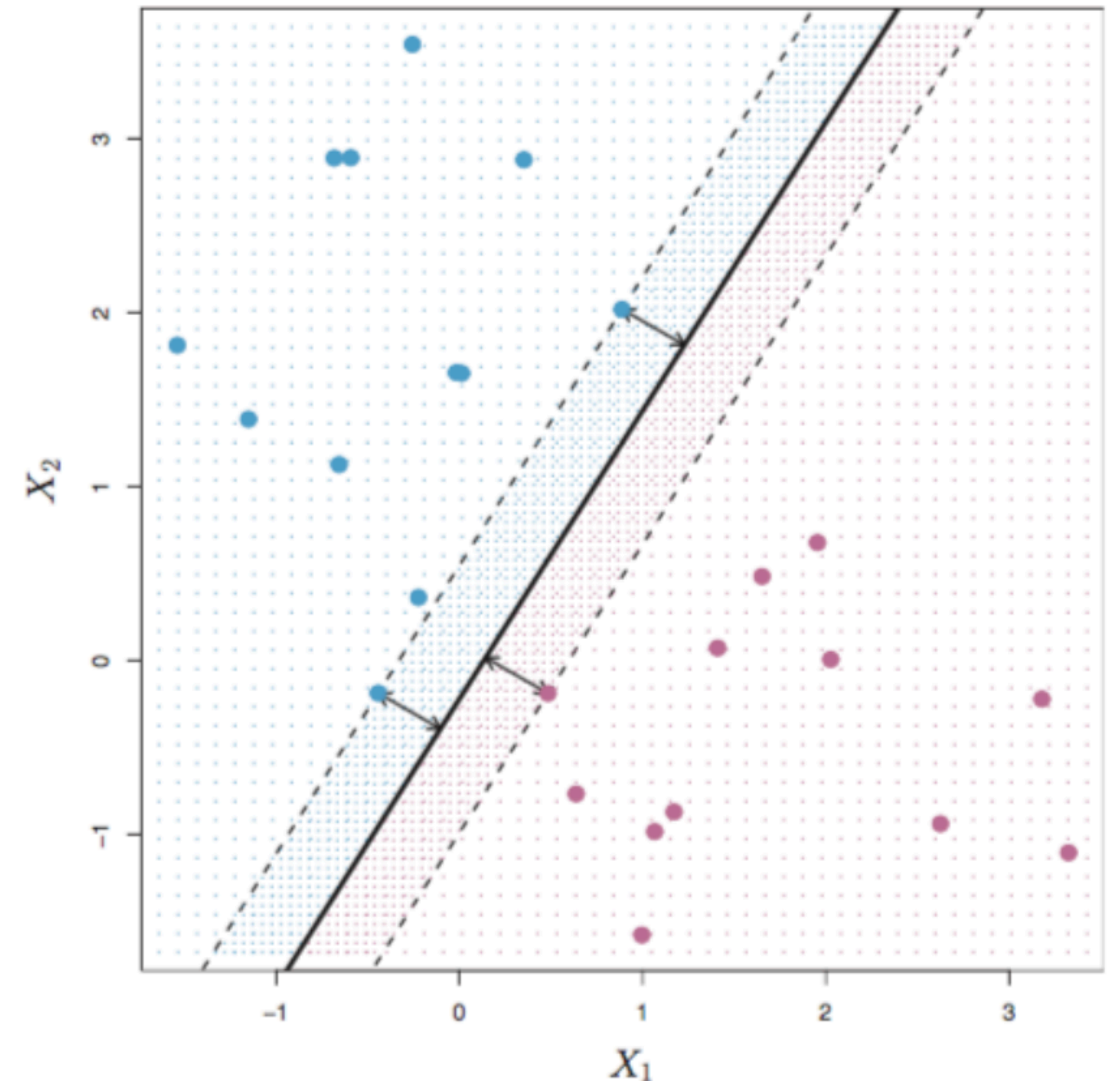
That's what motivates the **maximal margin classifier** — choosing the one that is farthest away from the data points.



# Maximal Margin Hyperplane

**Which** of the infinite separating hyperplanes should we choose?

A natural choice is the **maximal margin hyperplane**, the separating hyperplane that is farthest from the training samples.

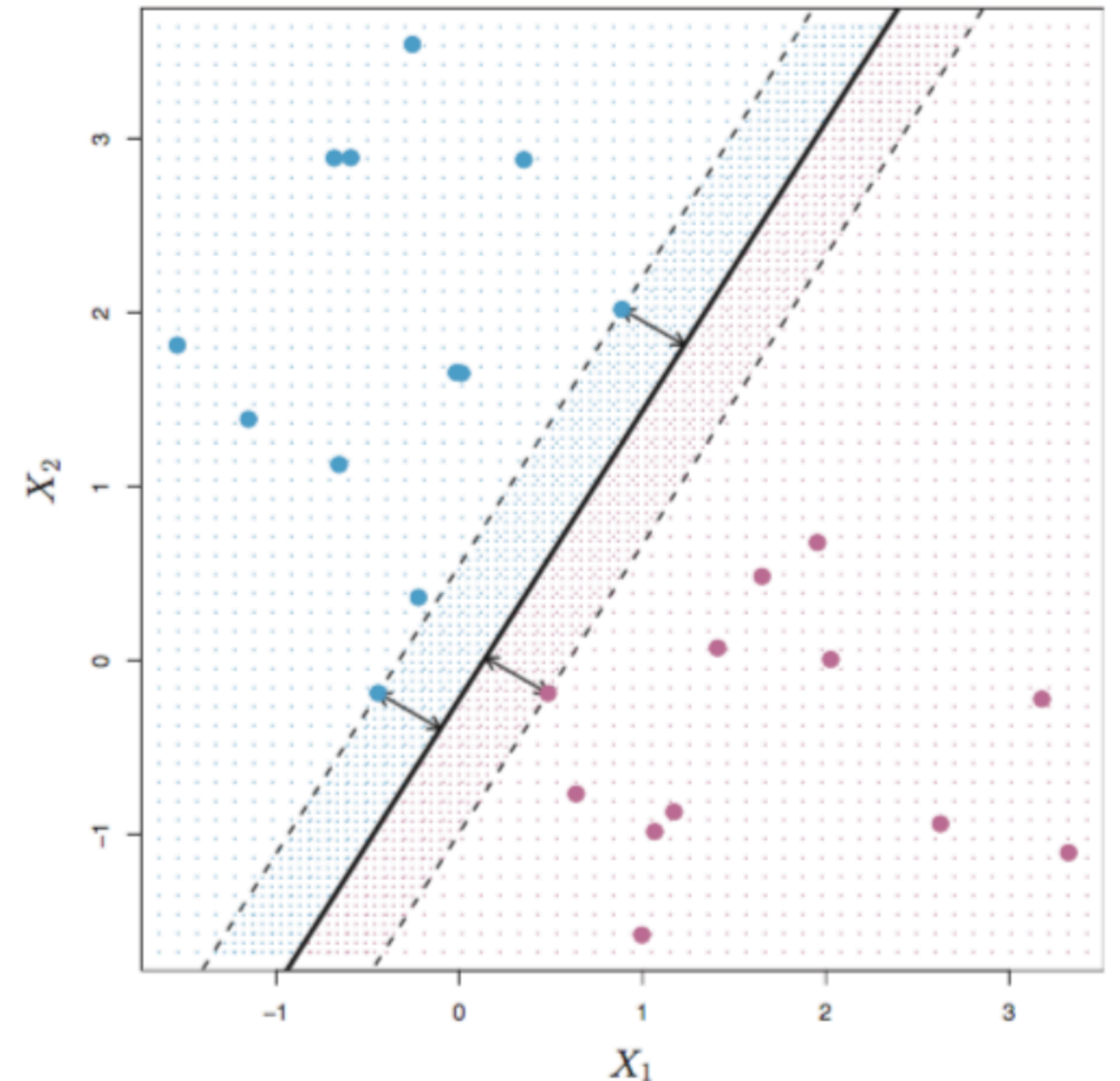


# Maximal Margin Hyperplane

## Problem Definition

**Margin:** smallest distance between any training observation and the hyperplane.

**Support vectors:** training observations whose distance to the hyperplane is equal to the margin.



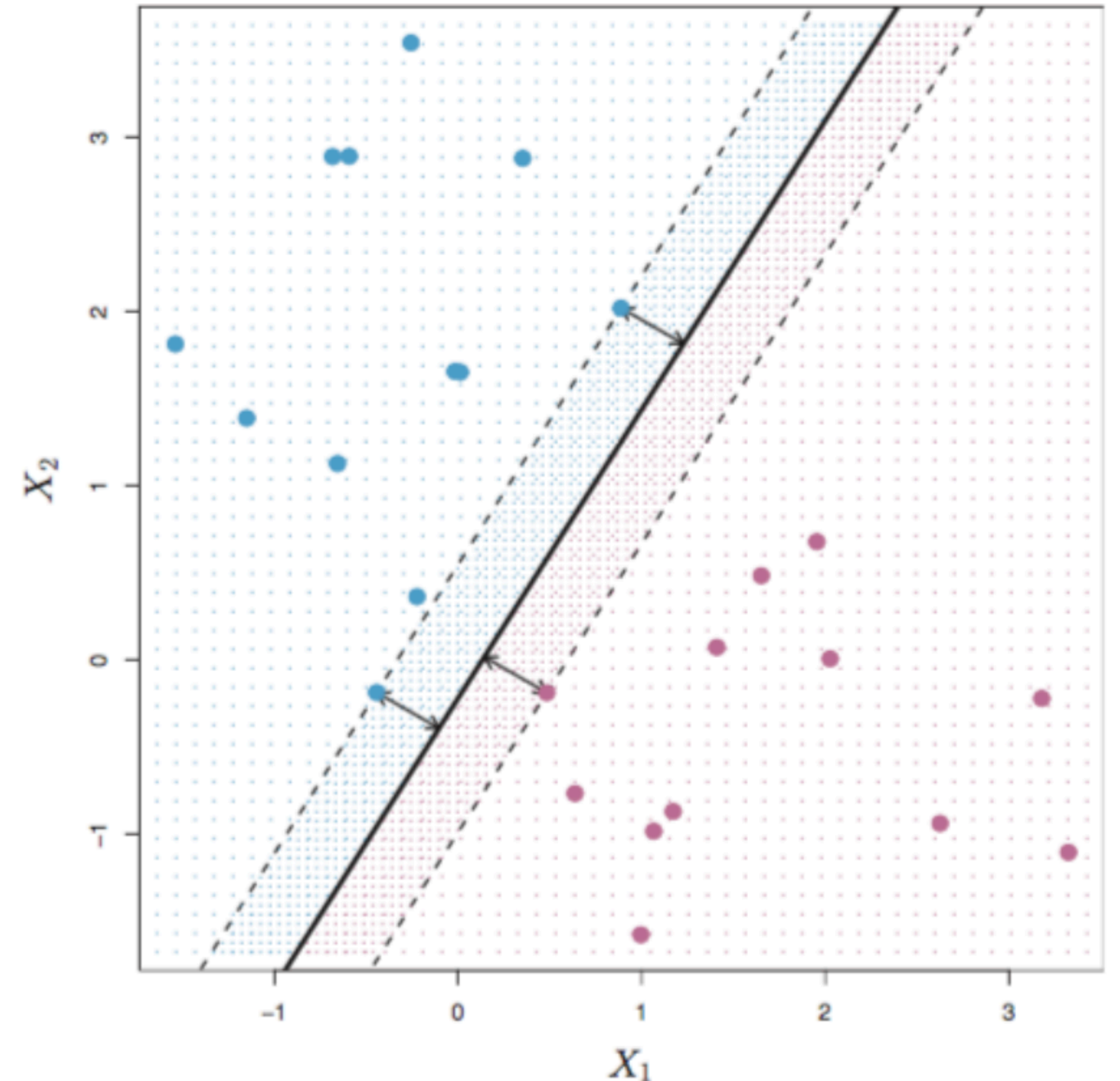
# Why Is It Called Support Vector?

The special points that sit right on the margin are called **support vectors**.

**Support:** maximal margin hyperplane only depends on these observations. if you remove or move one of these points, the boundary changes.

**Vector:** points are vectors in  $p$ -dimensional space.

If you move other points that are *not* support vectors, the boundary usually stays the same.



# Finding Maximal Margin Classifier

To find the maximal margin hyperplane on data  $(\vec{x}^{(i)}, y^{(i)}), \quad y^{(i)} \in \{-1, 1\}$

A  $p$  dimensions, hyperplane  $\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p = 0$

$$\max_{\beta_0, \dots, \beta_p} M$$

$$\text{subject to } \sum_{j=0}^p \beta_j^2 = 1$$

$$y^{(i)} (\beta_0 + \beta_1 x_1^{(i)} + \dots + \beta_p x_p^{(i)}) \geq M, \quad \forall i$$

# Finding Maximal Margin Classifier

To find the maximal margin hyperplane on data  $(\vec{x}^{(i)}, y^{(i)}), \quad y^{(i)} \in \{-1, 1\}$

A  $p$  dimensions, hyperplane  $\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p = 0$

$$\max_{\beta_0, \dots, \beta_p} M$$

Maximize the margin,  $M$

$$\text{subject to } \sum_{j=0}^p \beta_j^2 = 1$$

$$y^{(i)} (\beta_0 + \beta_1 x_1^{(i)} + \dots + \beta_p x_p^{(i)}) \geq M, \quad \forall i$$

# Finding Maximal Margin Classifier

To find the maximal margin hyperplane on data  $(\vec{x}^{(i)}, y^{(i)}), \quad y^{(i)} \in \{-1, 1\}$

A  $p$  dimensions, hyperplane  $\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p = 0$

$$\max_{\beta_0, \dots, \beta_p} M$$

$$\text{subject to } \sum_{j=0}^p \beta_j^2 = 1$$

constraint necessary for well- defined  
optimization problem

$$y^{(i)} (\beta_0 + \beta_1 x_1^{(i)} + \dots + \beta_p x_p^{(i)}) \geq M, \quad \forall i$$

# Finding Maximal Margin Classifier

To find the maximal margin hyperplane on data  $(\vec{x}^{(i)}, y^{(i)}), \quad y^{(i)} \in \{-1, 1\}$

A  $p$  dimensions, hyperplane  $\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p = 0$

$$\max_{\beta_0, \dots, \beta_p} M$$

$$\text{subject to } \sum_{j=0}^p \beta_j^2 = 1$$

constraint necessary for well-defined optimization problem

$$y^{(i)} (\beta_0 + \beta_1 x_1^{(i)} + \dots + \beta_p x_p^{(i)}) \geq M, \quad \forall i$$

all training points must be at least distance  $M$  from hyperplane

# Finding Maximal Margin Classifier

To find the maximal margin hyperplane on data  $(\vec{x}^{(i)}, y^{(i)}), \quad y^{(i)} \in \{-1, 1\}$

A  $p$  dimensions, hyperplane  $\beta_0 + \beta_1 X_1 + \dots + \beta_p X_p = 0$

$$\max_{\beta_0, \dots, \beta_p} M$$

$$\text{subject to } \sum_{j=0}^p \beta_j^2 = 1$$

$$y^{(i)} (\beta_0 + \beta_1 x_1^{(i)} + \dots + \beta_p x_p^{(i)}) \geq M, \quad \forall i$$

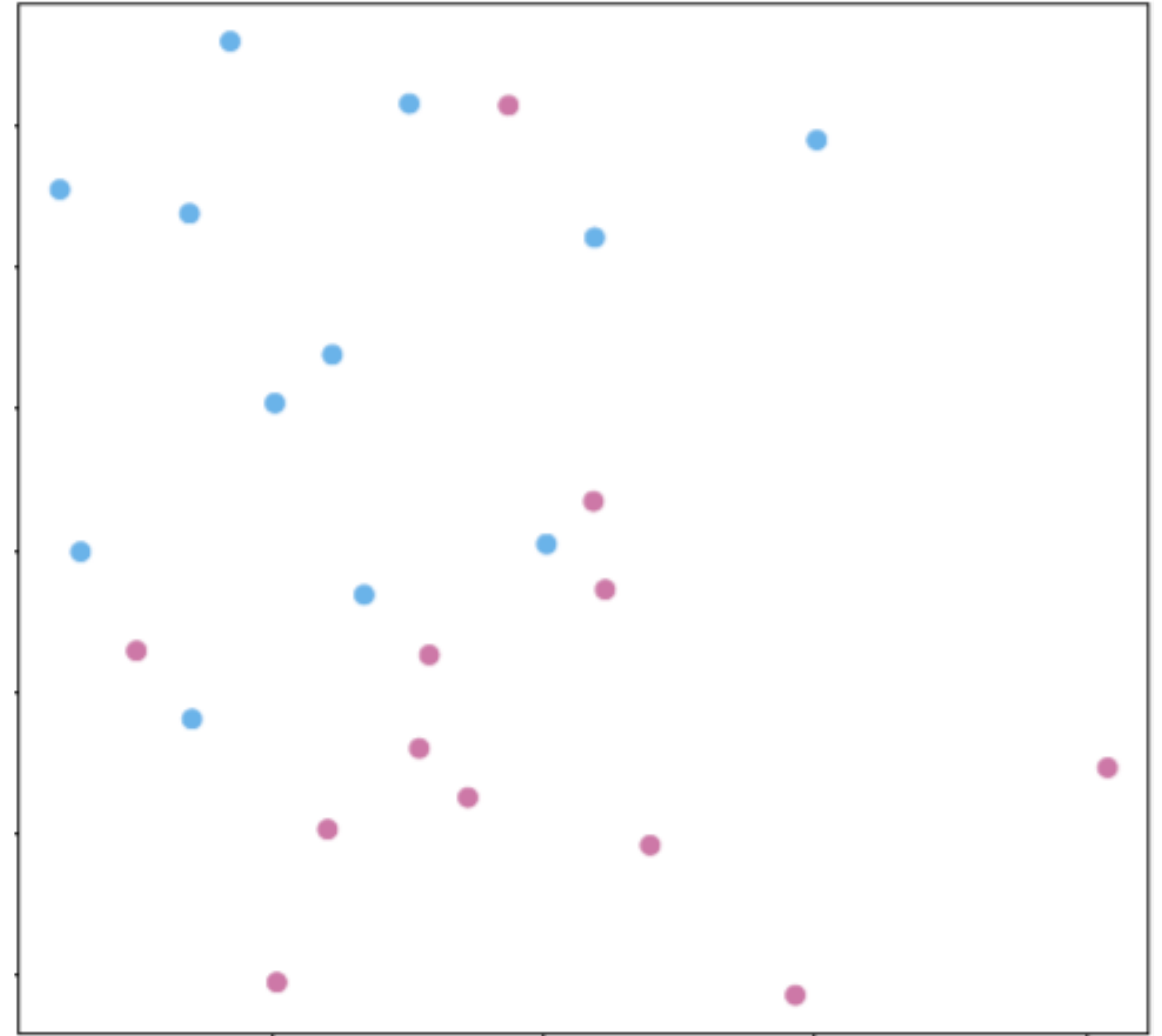
This Can be written as a convex optimization problem.

We know how to solve convex optimization problems efficiently to find  $M$  and  $\beta$ .

# Maximal Margin Classifier

Recall the **assumption**: Classes can be separated by a linear decision boundary.

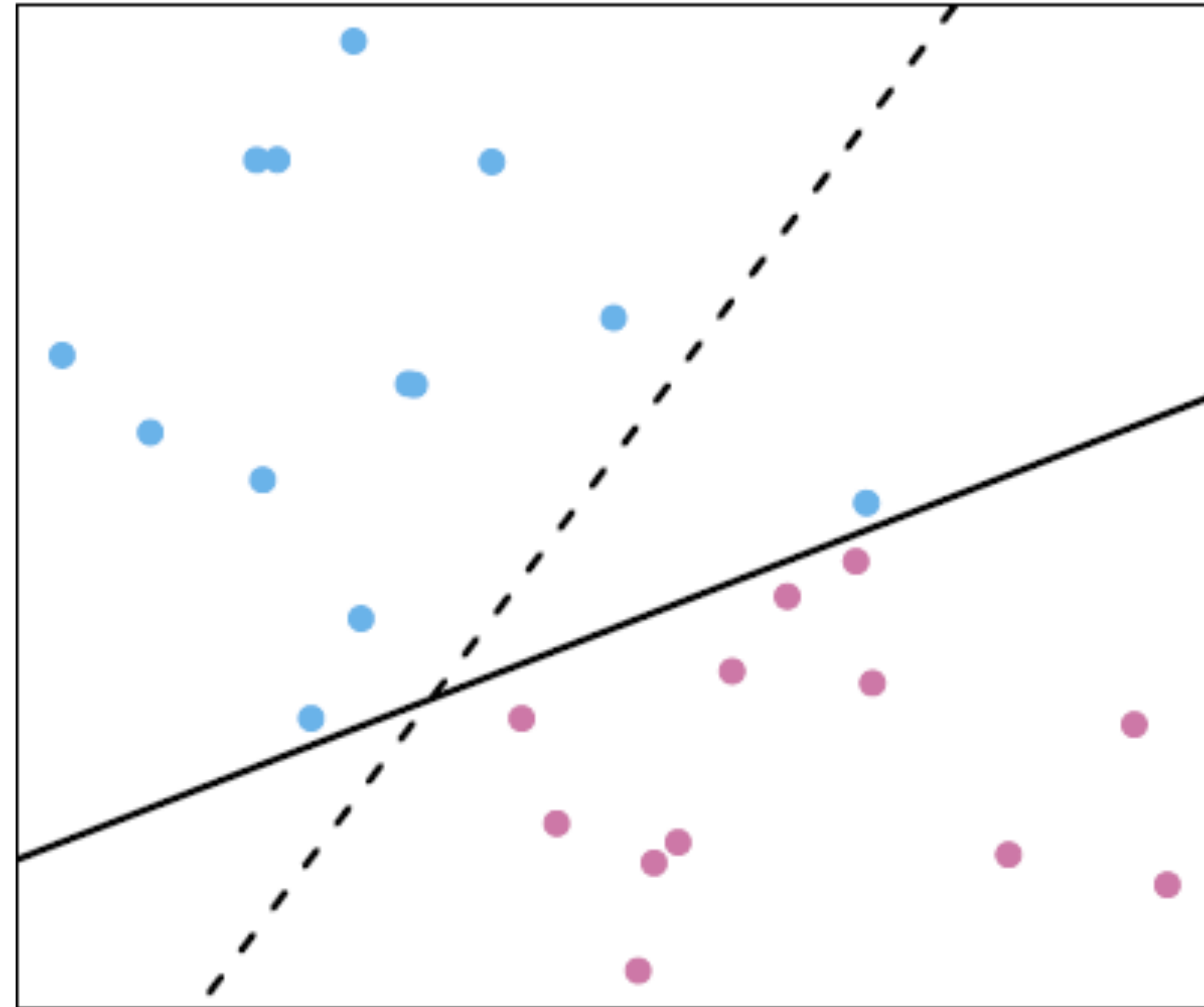
# What if there is no separating hyperplane?



# Maximal Margin Classifier

**Disadvantage** of the maximal margin classifier:

- Can be sensitive to individual observations
- May overfit training data



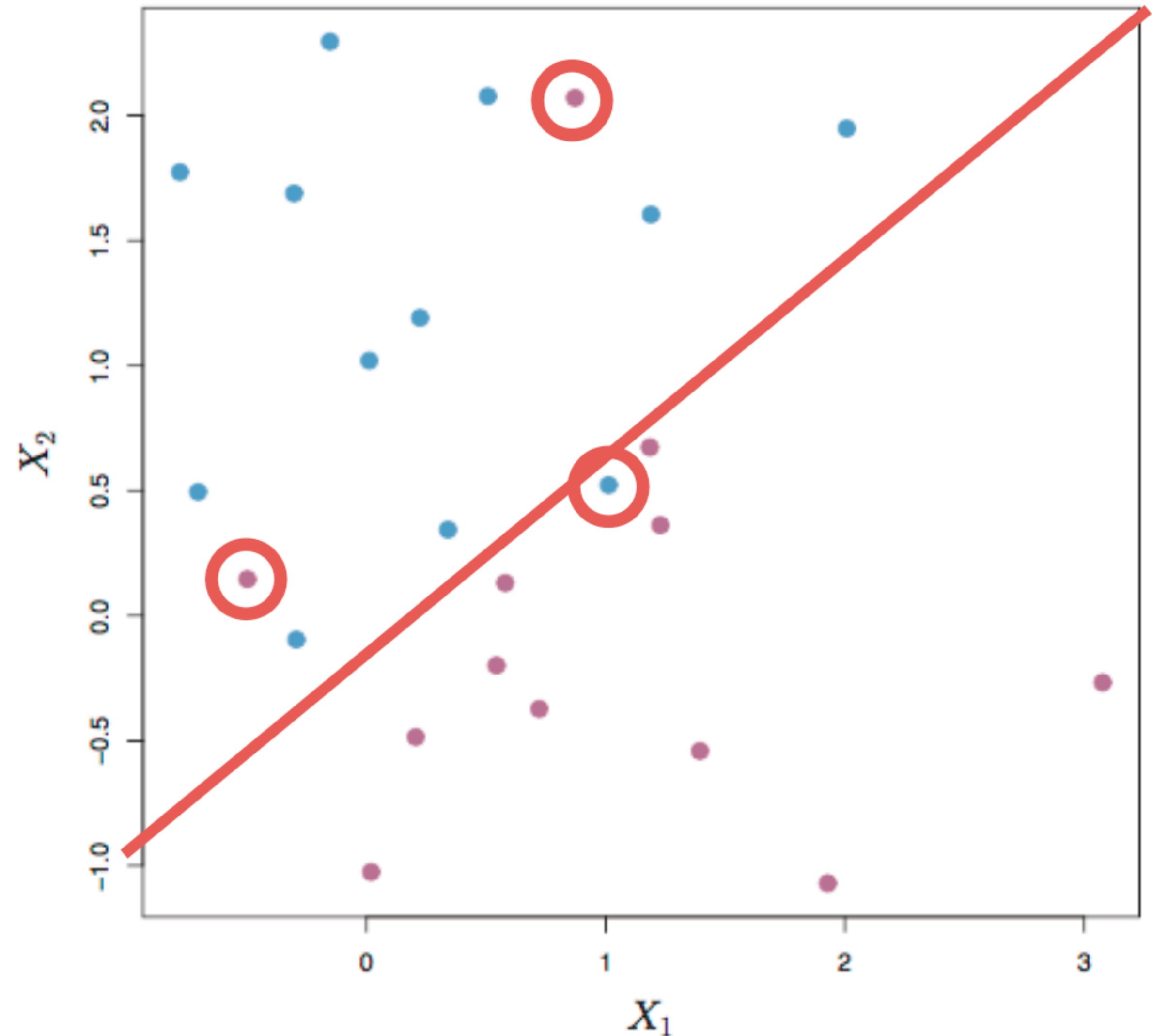
# Support Vector Classifier

# Support vector Classifier

Like the maximal margin classifier, it looks for a hyperplane to perform classification.

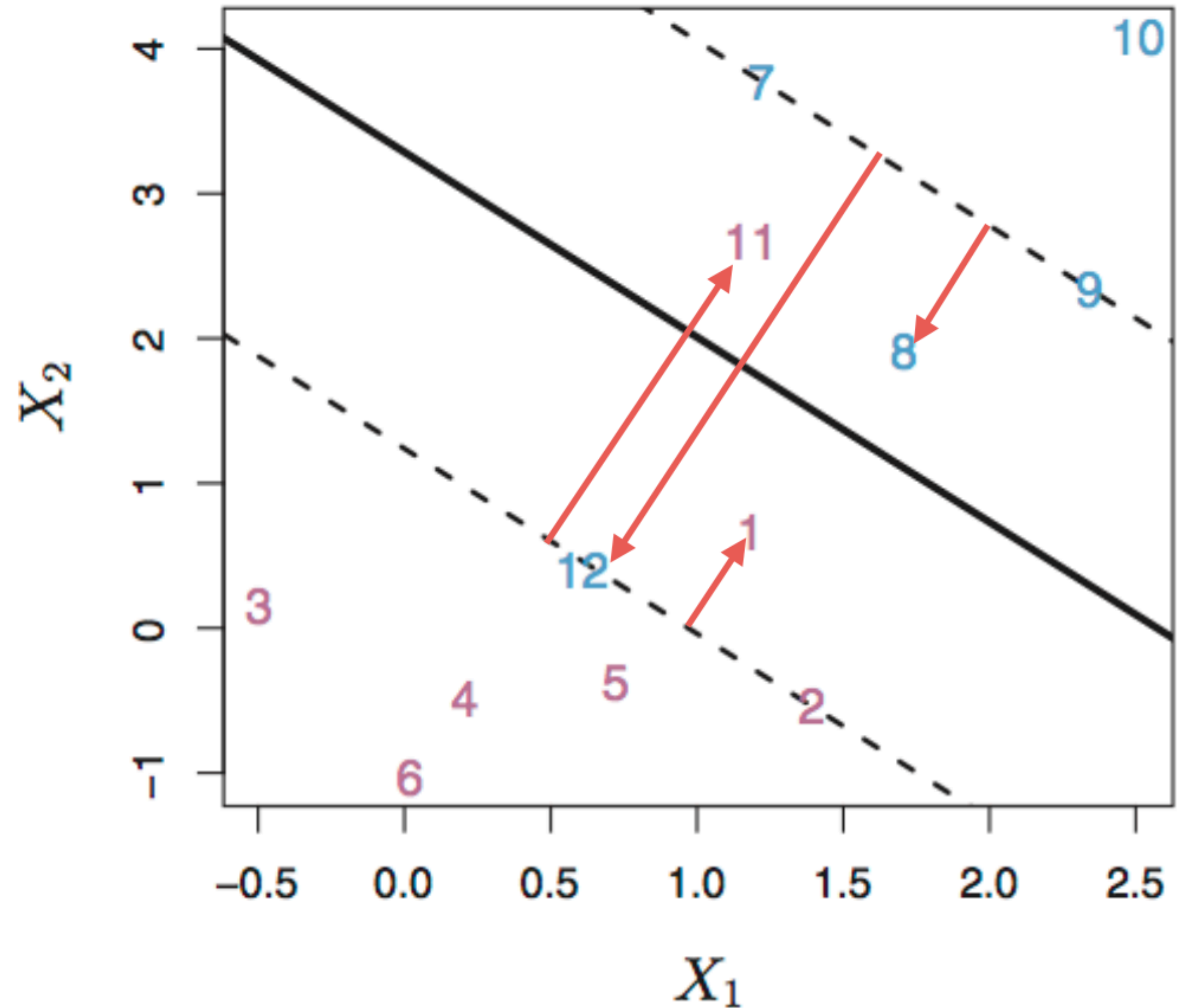
However, training samples are allowed to be on the “wrong side” of the margin or hyperplane.

This hyperplane *almost* separates the classes using a “**soft margin**”.



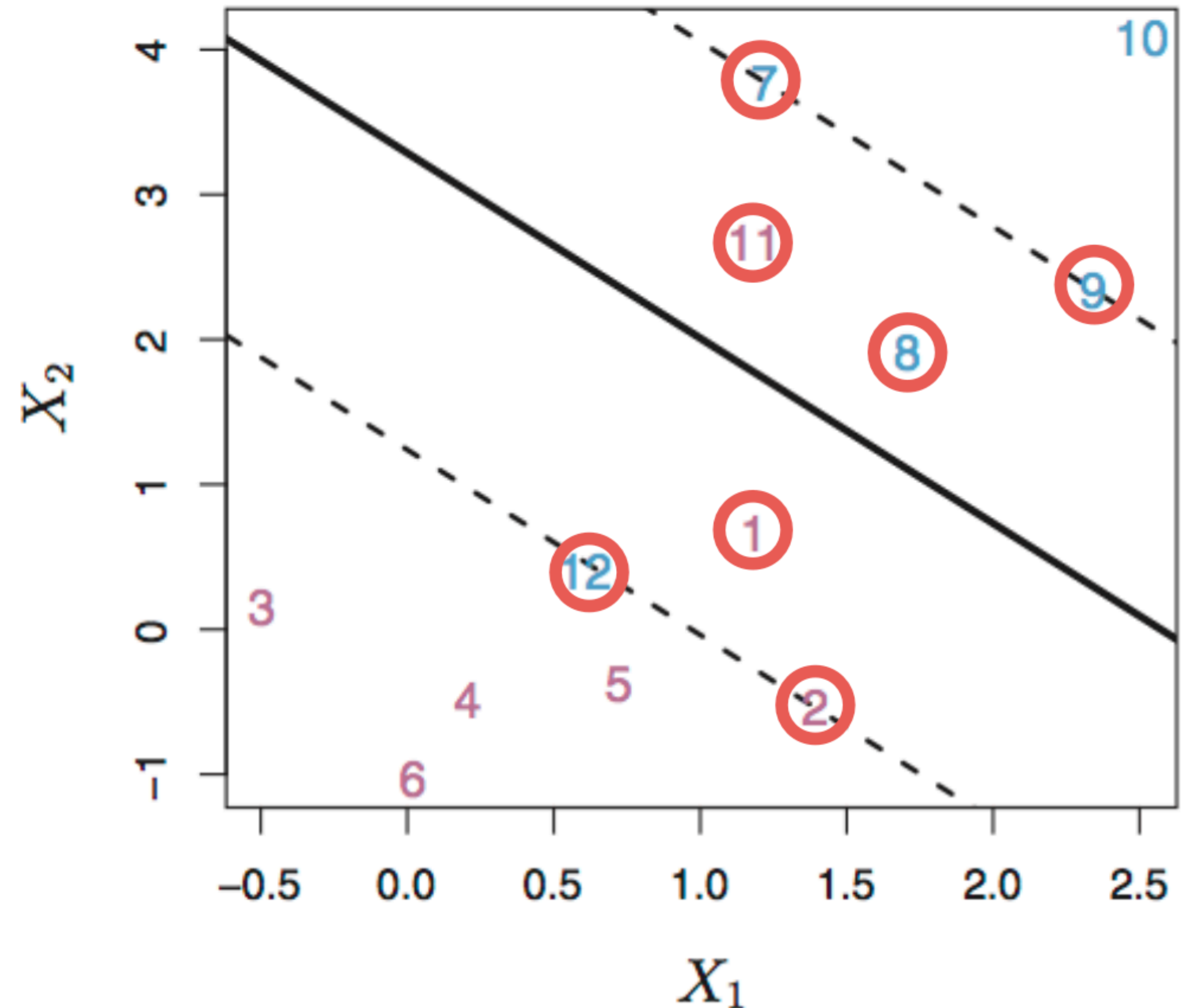
# Support vector Classifier

Some points are allowed to violate the margin.



# Support vector Classifier

- Some points are allowed to violate the margin.
- Support vector classifiers also have support vectors.
- They are points lying directly on the margin, or on the wrong side of the margin for their class.
- These observations affect the hyperplane.



# Finding Support vector Classifier

To find the support vector classifier hyperplane

$$\max_{\beta_0, \dots, \beta_p, \epsilon_1, \dots, \epsilon_n} M$$

Maximize the margin, M

$$\text{subject to } \sum_{j=0}^p \beta_j^2 = 1$$

constraint necessary for well- defined optimization problem

$$y^{(i)} (\beta_0 + \beta_1 x_1^{(i)} + \dots + \beta_p x_p^{(i)}) \geq M(1 - \epsilon_i), \quad \forall i$$

$$\sum_{i=1}^n \epsilon_i \leq C, \quad \epsilon_i \geq 0, \quad \forall i$$

# Finding Support vector Classifier

To find the support vector classifier hyperplane

$$\max_{\beta_0, \dots, \beta_p, \epsilon_1, \dots, \epsilon_n} M$$

$$\text{subject to } \sum_{j=0}^p \beta_j^2 = 1$$

training points must be at least distance  $M$   
from hyperplane,  
or pay a penalty  $\epsilon_i$

$$y^{(i)} (\beta_0 + \beta_1 x_1^{(i)} + \dots + \beta_p x_p^{(i)}) \geq M(1 - \epsilon_i), \quad \forall i$$

$$\sum_{i=1}^n \epsilon_i \leq C, \quad \epsilon_i \geq 0, \quad \forall i$$

# Finding Support vector Classifier

To find the support vector classifier hyperplane

$$\max_{\beta_0, \dots, \beta_p, \epsilon_1, \dots, \epsilon_n} M$$

$$\text{subject to } \sum_{j=0}^p \beta_j^2 = 1$$

training points must be at least distance  $M$   
from hyperplane,  
or pay a penalty  $\epsilon_i$

$$y^{(i)} (\beta_0 + \beta_1 x_1^{(i)} + \dots + \beta_p x_p^{(i)}) \geq M(1 - \epsilon_i), \quad \forall i$$

$$\sum_{i=1}^n \epsilon_i \leq C,$$

$$\epsilon_i \geq 0, \quad \forall i$$

“slack” variable  $\epsilon_i$

# Finding Support vector Classifier

To find the support vector classifier hyperplane

$$\max_{\beta_0, \dots, \beta_p, \epsilon_1, \dots, \epsilon_n} M$$

$$\text{subject to } \sum_{j=0}^p \beta_j^2 = 1$$

$$y^{(i)}(\beta_0 + \beta_1 x_1^{(i)} + \dots + \beta_p x_p^{(i)}) \geq M(1 - \epsilon_i), \quad \forall i$$

$$\sum_{i=1}^n \epsilon_i \leq C,$$

$$\epsilon_i \geq 0, \quad \forall i$$

“slack” variable  $\epsilon_i$

The amount of violation a point makes

- $\epsilon_i = 0 \rightarrow$  the point is correctly placed outside the margin.
- $0 < \epsilon_i \leq 1 \rightarrow$  the point is inside the margin but still on the correct side.
- $\epsilon_i > 1 \rightarrow$  the point is actually misclassified (on the wrong side of the hyperplane).

# Finding Support vector Classifier

To find the support vector classifier hyperplane

$$\max_{\beta_0, \dots, \beta_p, \epsilon_1, \dots, \epsilon_n} M$$

$$\text{subject to } \sum_{j=0}^p \beta_j^2 = 1$$

$$y^{(i)}(\beta_0 + \beta_1 x_1^{(i)} + \dots + \beta_p x_p^{(i)}) \geq M(1 - \epsilon_i), \quad \forall i$$

$$\sum_{i=1}^n \epsilon_i \leq C, \quad \epsilon_i \geq 0, \quad \forall i$$

Penalty parameter

“slack” variable  $\epsilon_i$

- $\epsilon_i$  how much a point breaks the margin rule.
- we don't want *too many* violations, so we put a **budget limit** on them.
- **C — the penalty parameter:** In other words, you can violate the margin, but only by a total amount  $C$  on your entire dataset.

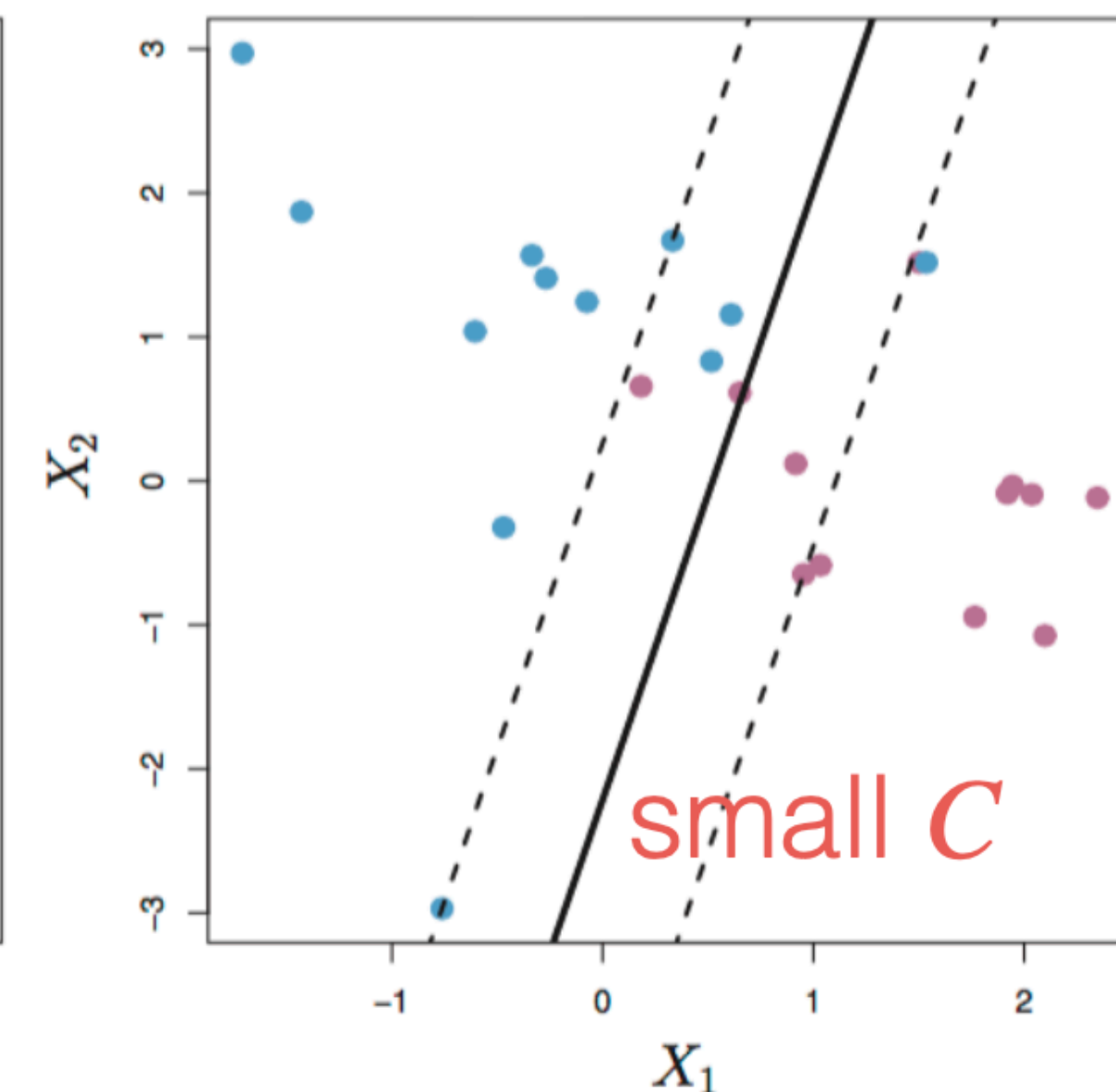
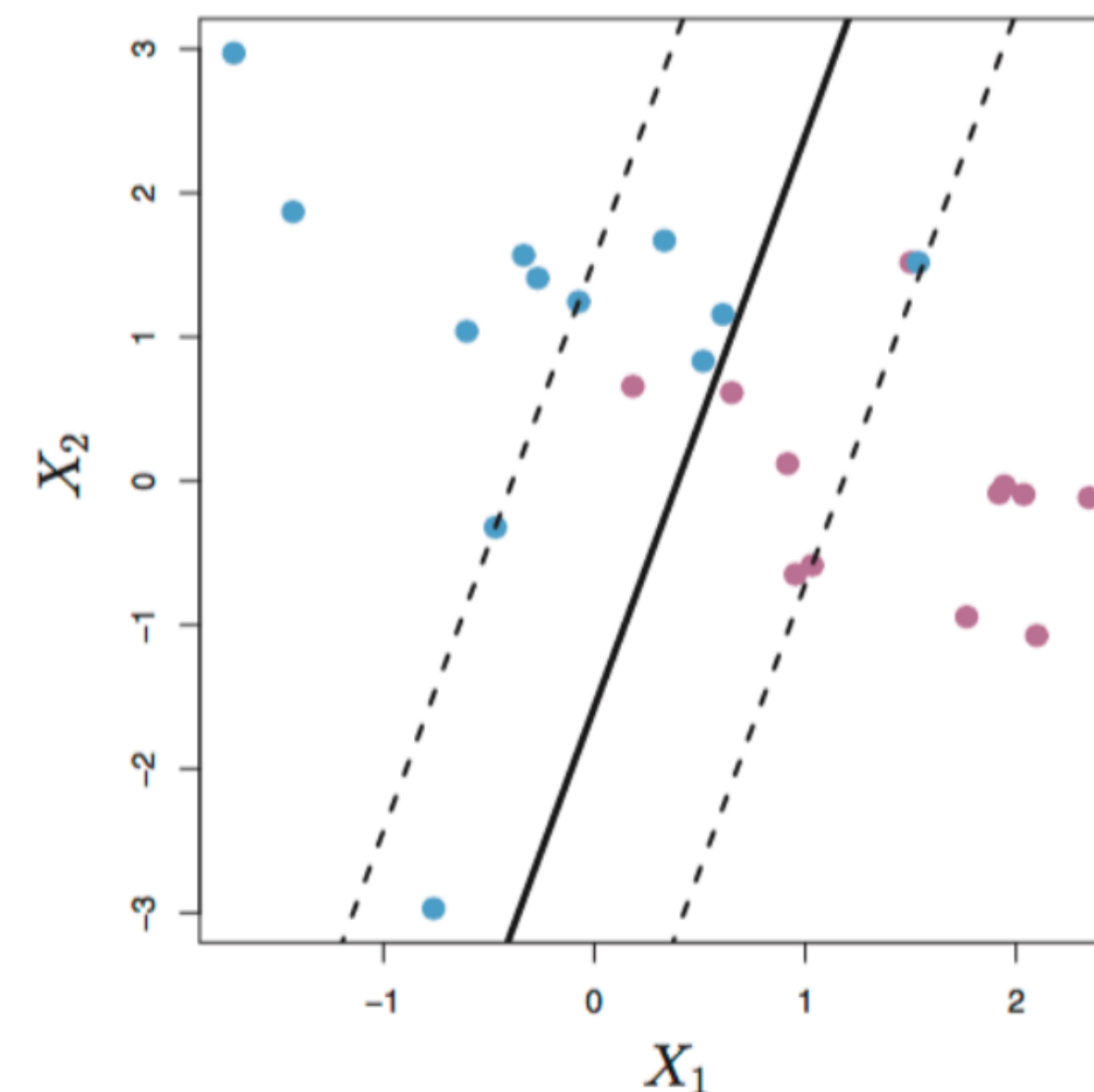
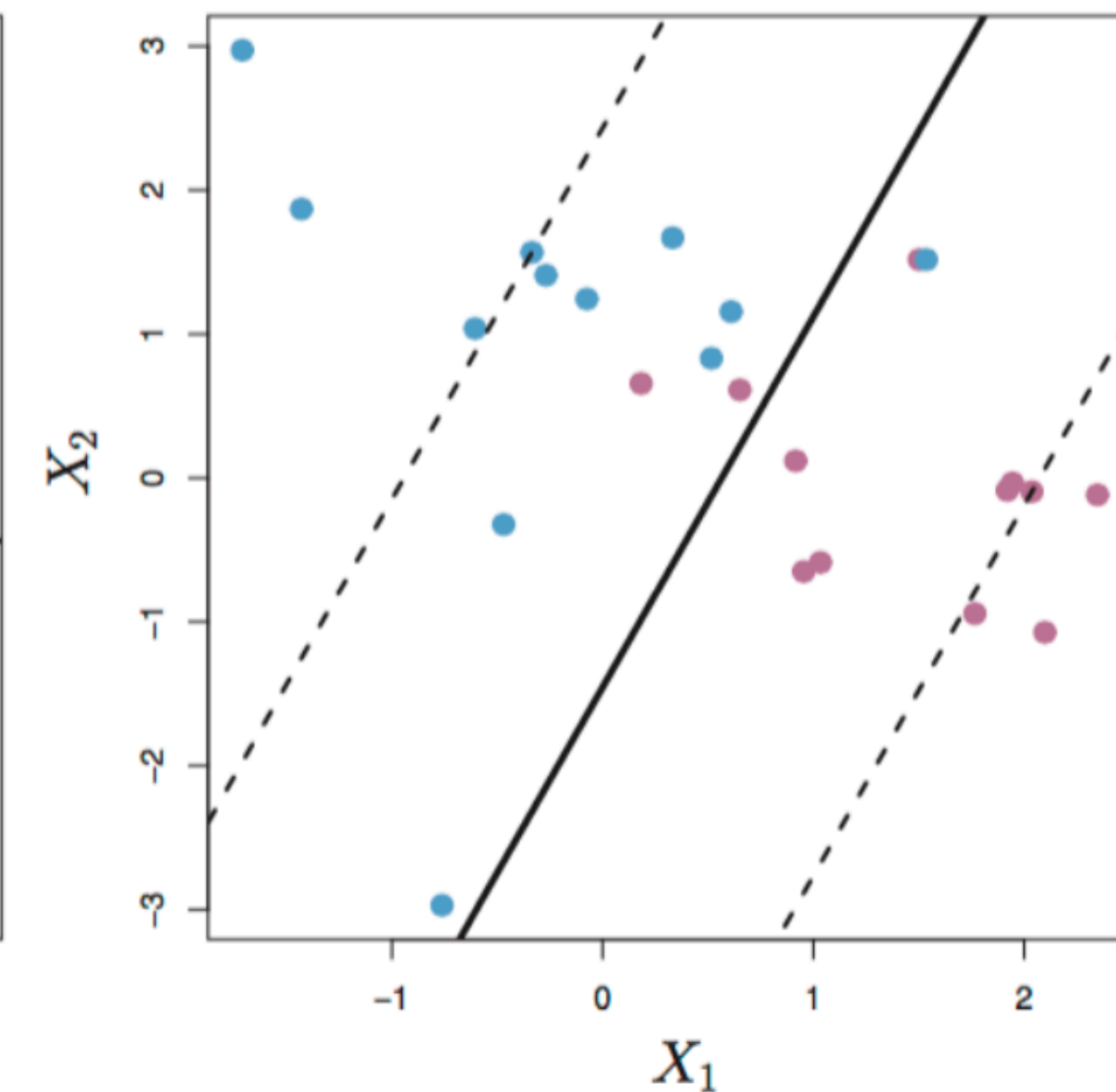
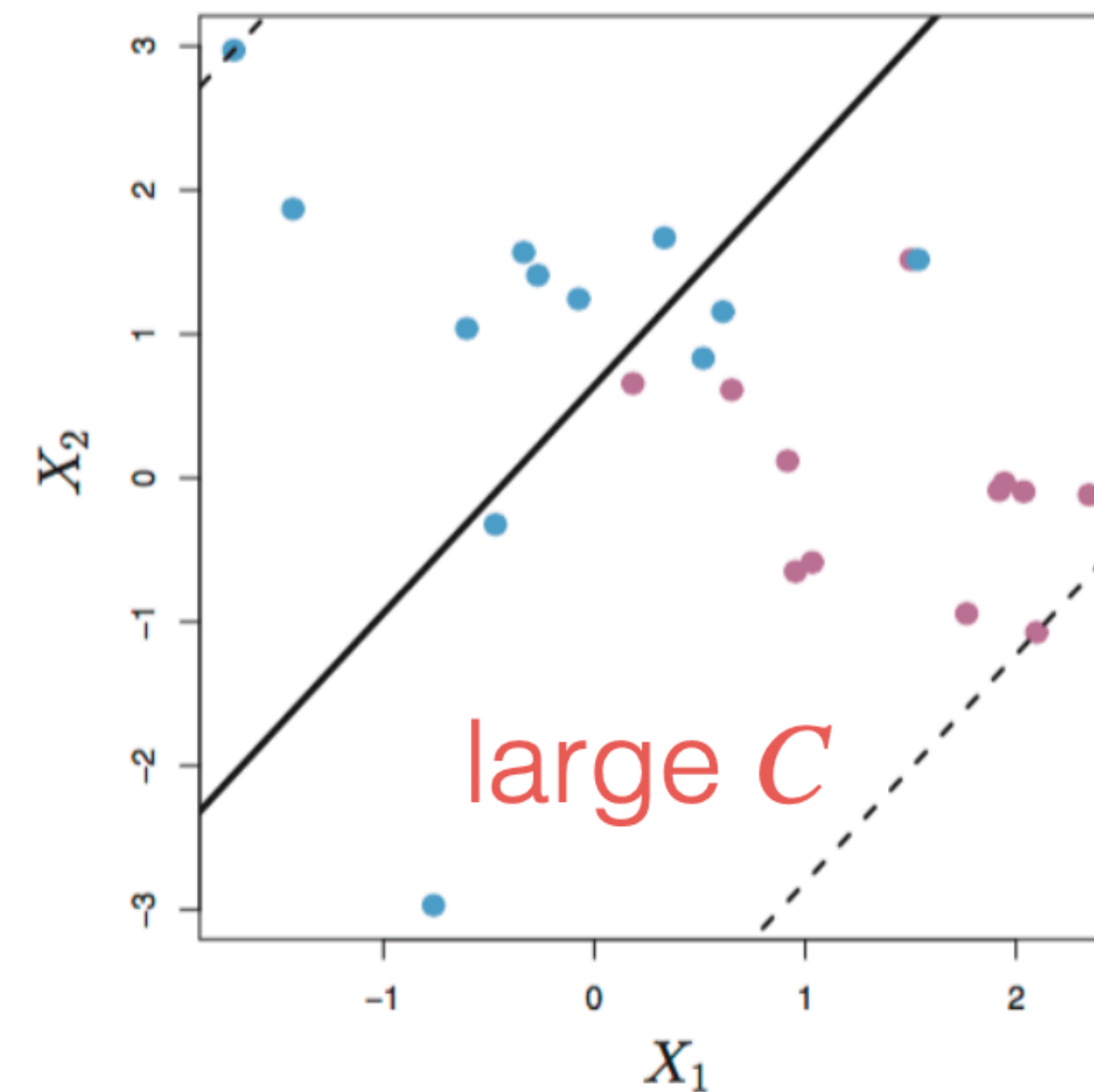
# How Do We Choose $C$ ?

- As with many things we don't know *a priori* in machine learning,  $C$  is a hyperparameter that we tune using cross-validation.
- Note that it must be non-negative.
- If  $C = 0$ , we recover the maximal margin classifier (if one exists).
- As  $C$  goes from small to large, there is a bias-variance tradeoff.
- Different text books or coding libraries have different definition for  $C$ . For example:
  - In ISLR,  $C$  is defined as a "**budget**" for violations.
  - In scikit-learn / LibSVM Convention (Standard Code)  $C$  is defined as a "**penalty parameter**".

# Bias — Variance & Parameter C

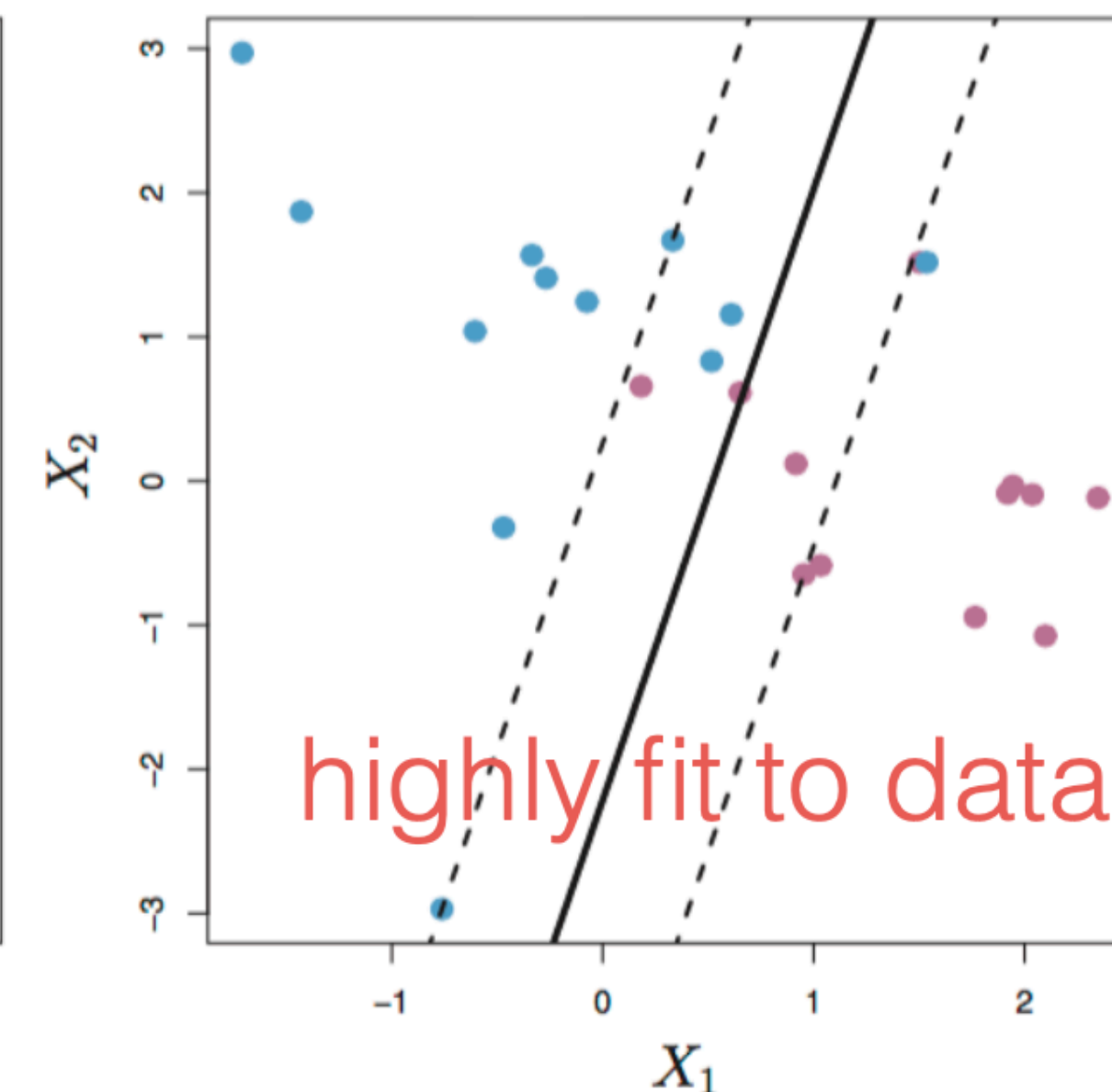
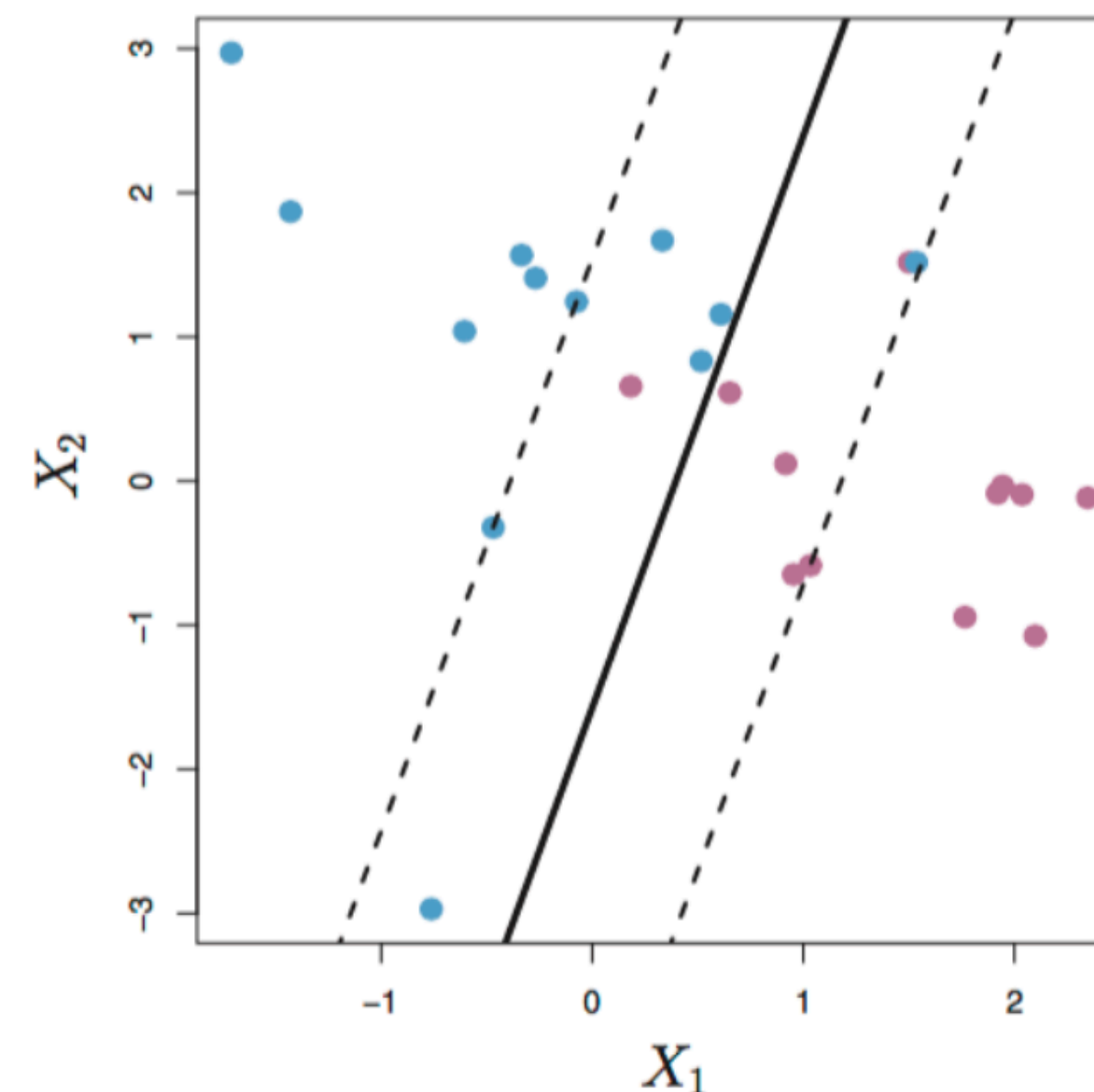
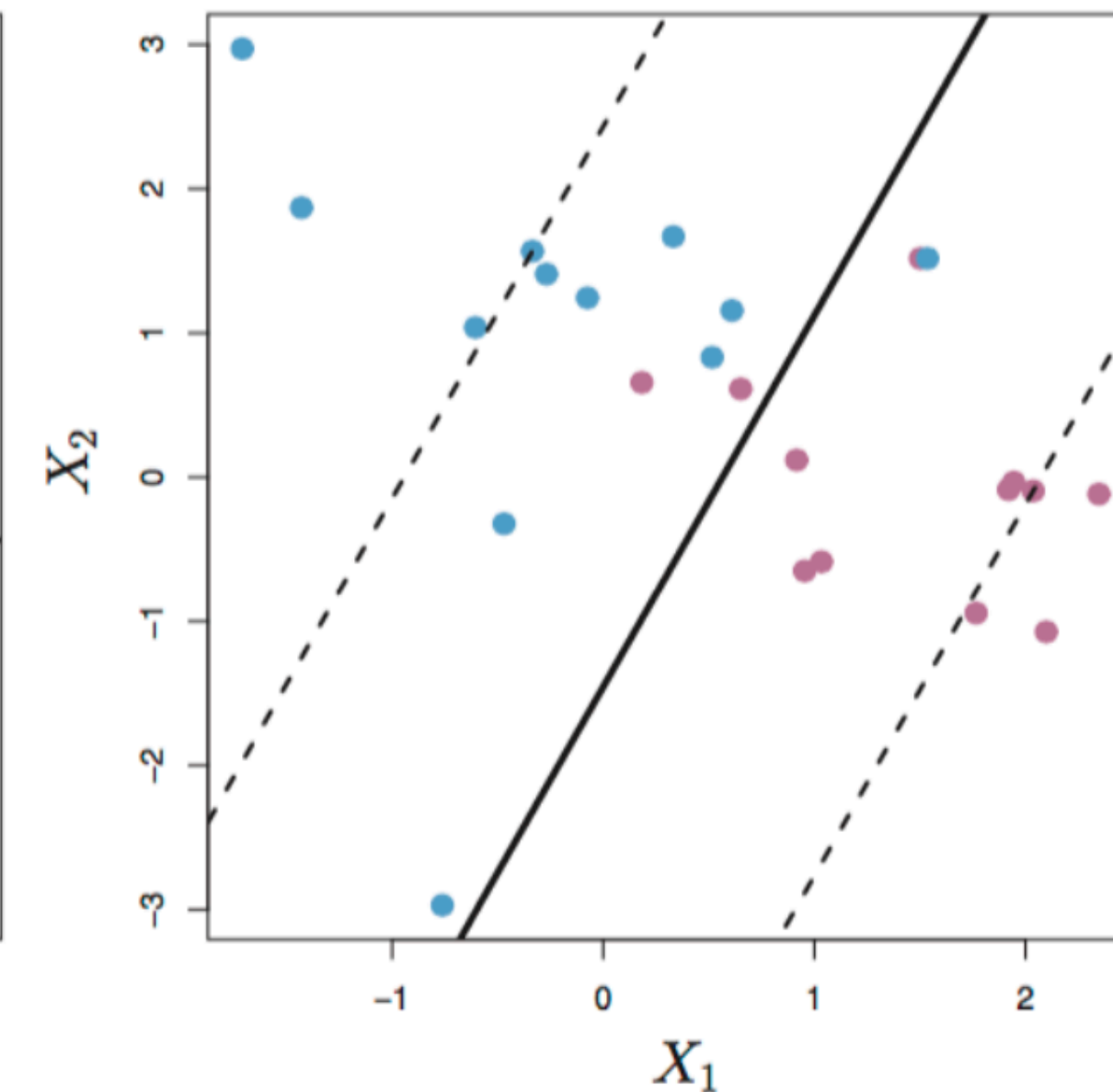
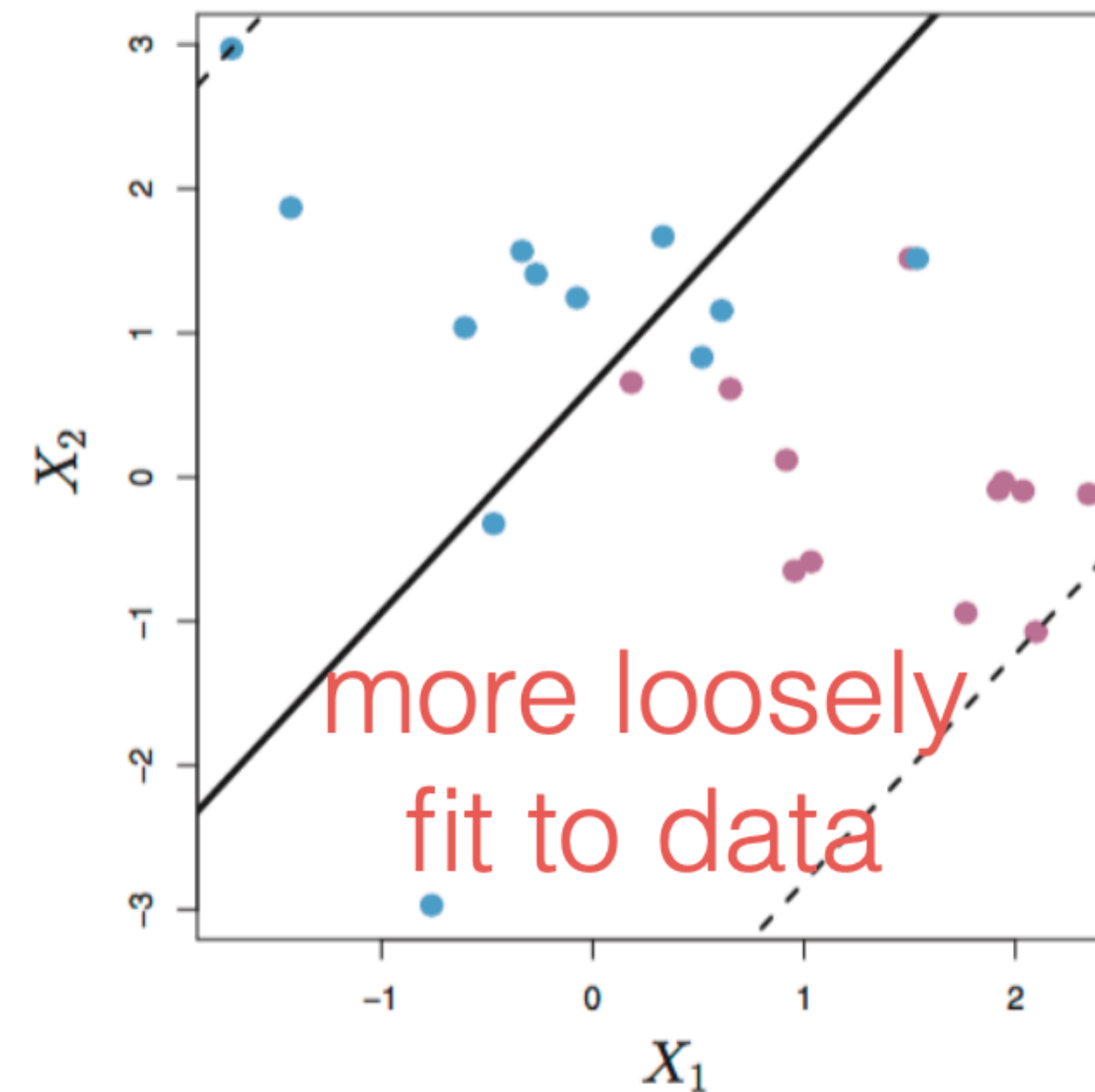
textbook: *An Introduction to Statistical Learning (ISLR)*

- Large  $C$ 
  - Large violation budget - Large margin
  - Many support vectors
- Small  $C$ 
  - Small violation budget - Small margin
  - Few support vectors



# Bias — Variance & Parameter C

- **Bias** = Error from simplifying too much.
- **Variance** = Error from being too sensitive to small changes in the training data.
- Large  $C$ : *You have a large budget for errors. This allows for a wider margin that ignores more individual points, leading to:*
  - High bias
  - Low variance



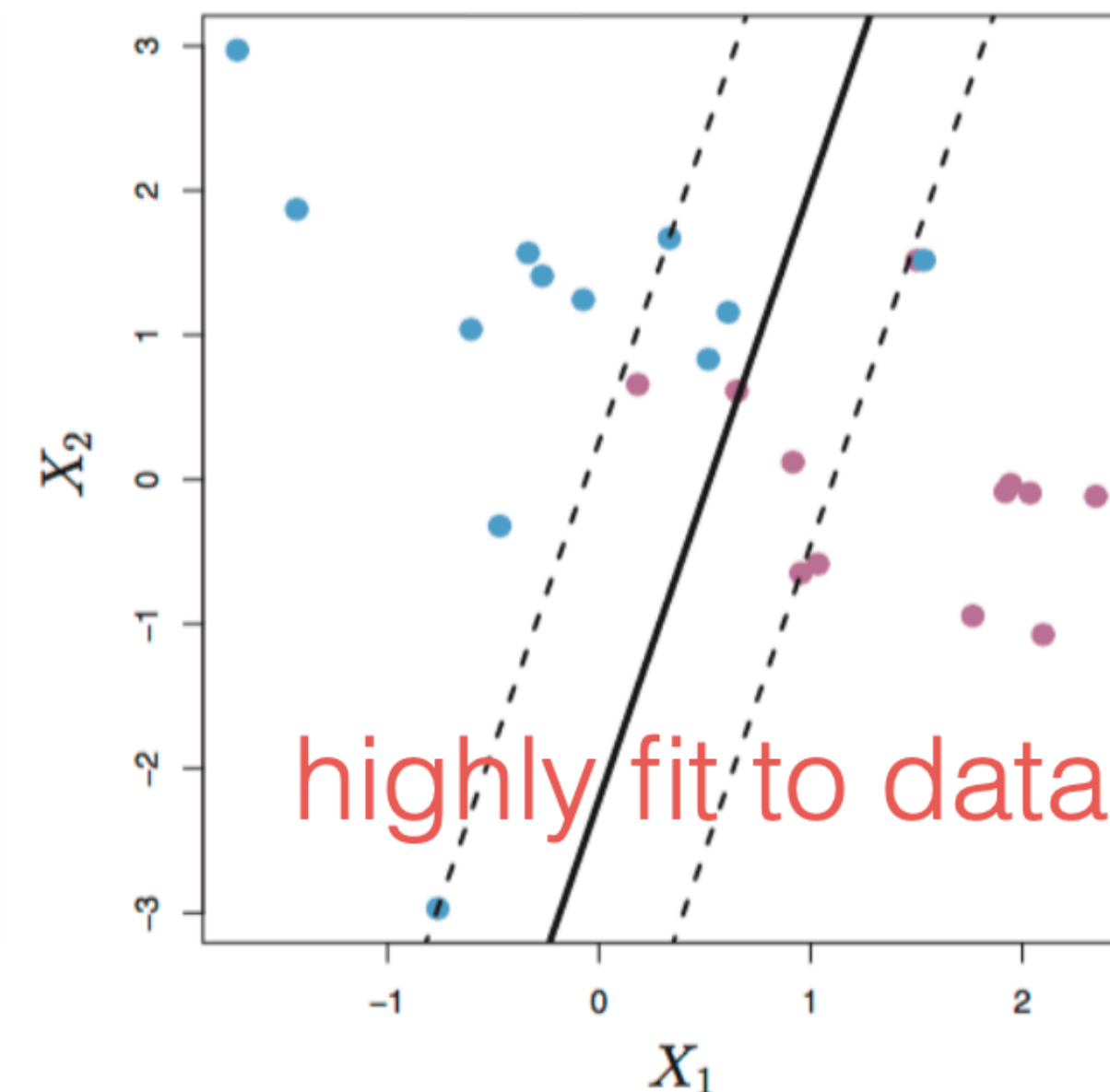
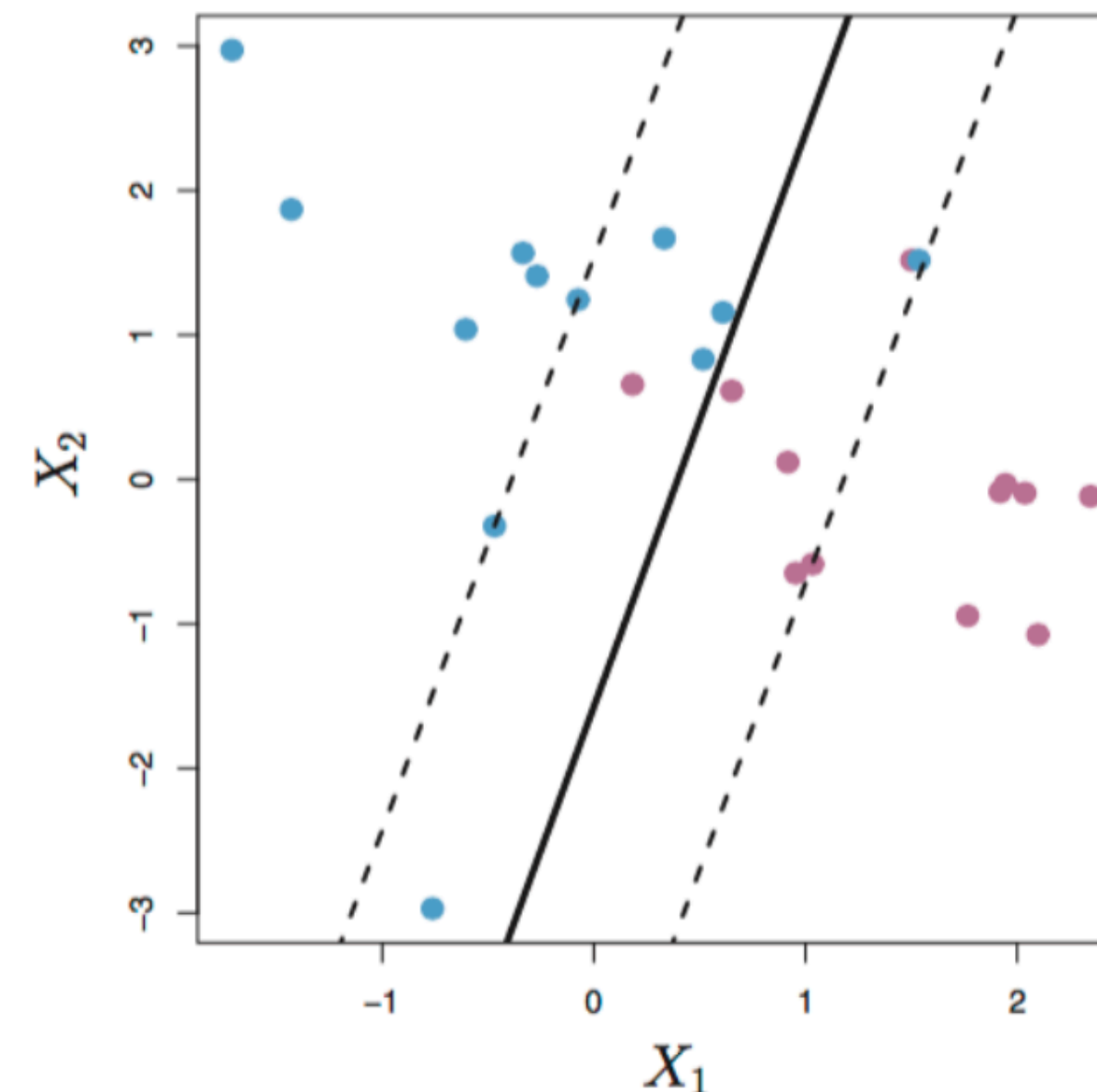
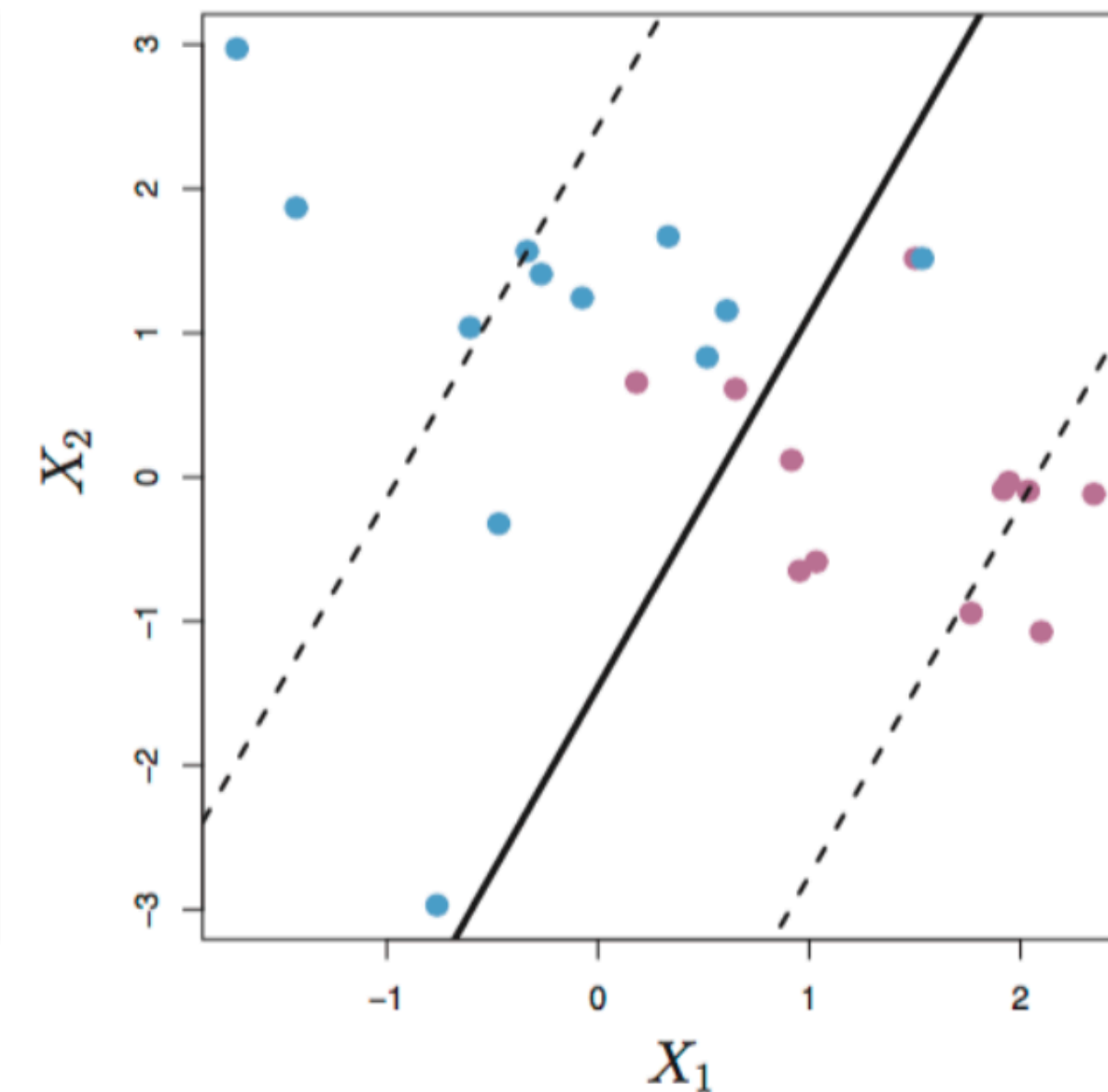
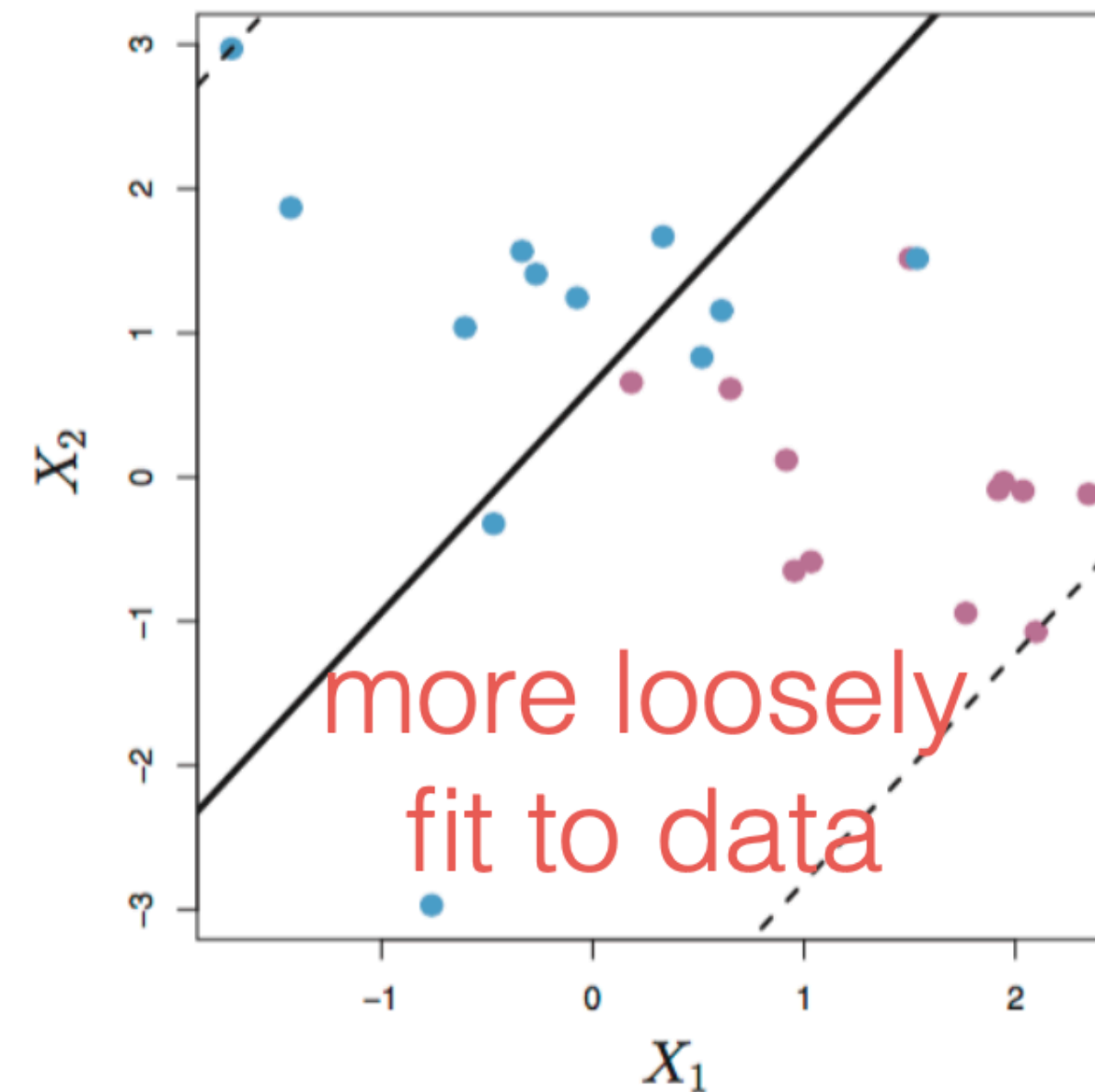
# Bias — Variance & Parameter C

- **Bias** = Error from simplifying too much.
- **Variance** = Error from being too sensitive to small changes in the training data.

- Small  $C$ : *You have a tiny budget for errors.*

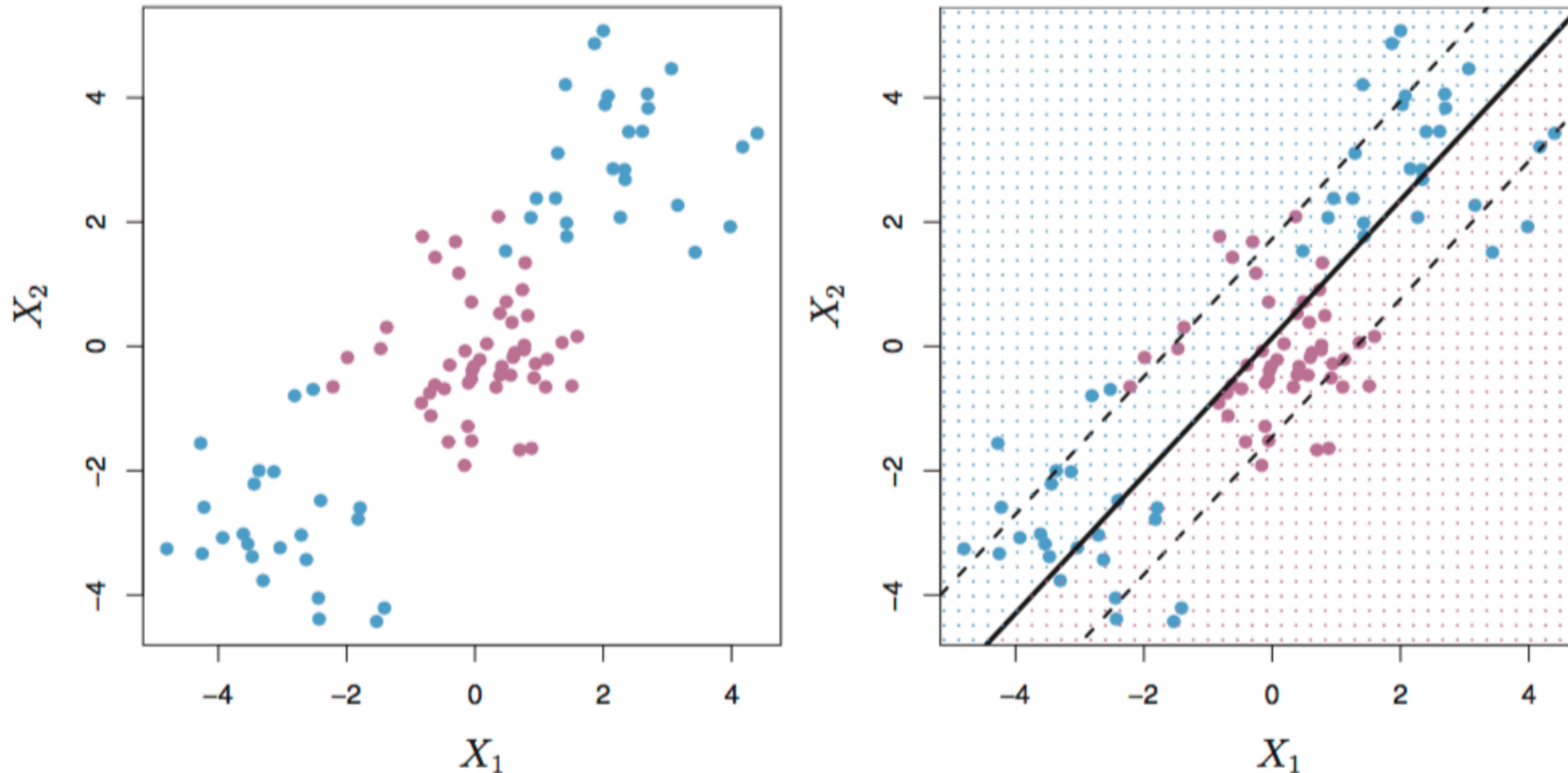
*The margin must be narrow and strict to avoid violations, leading to:*

- Low bias
- High variance



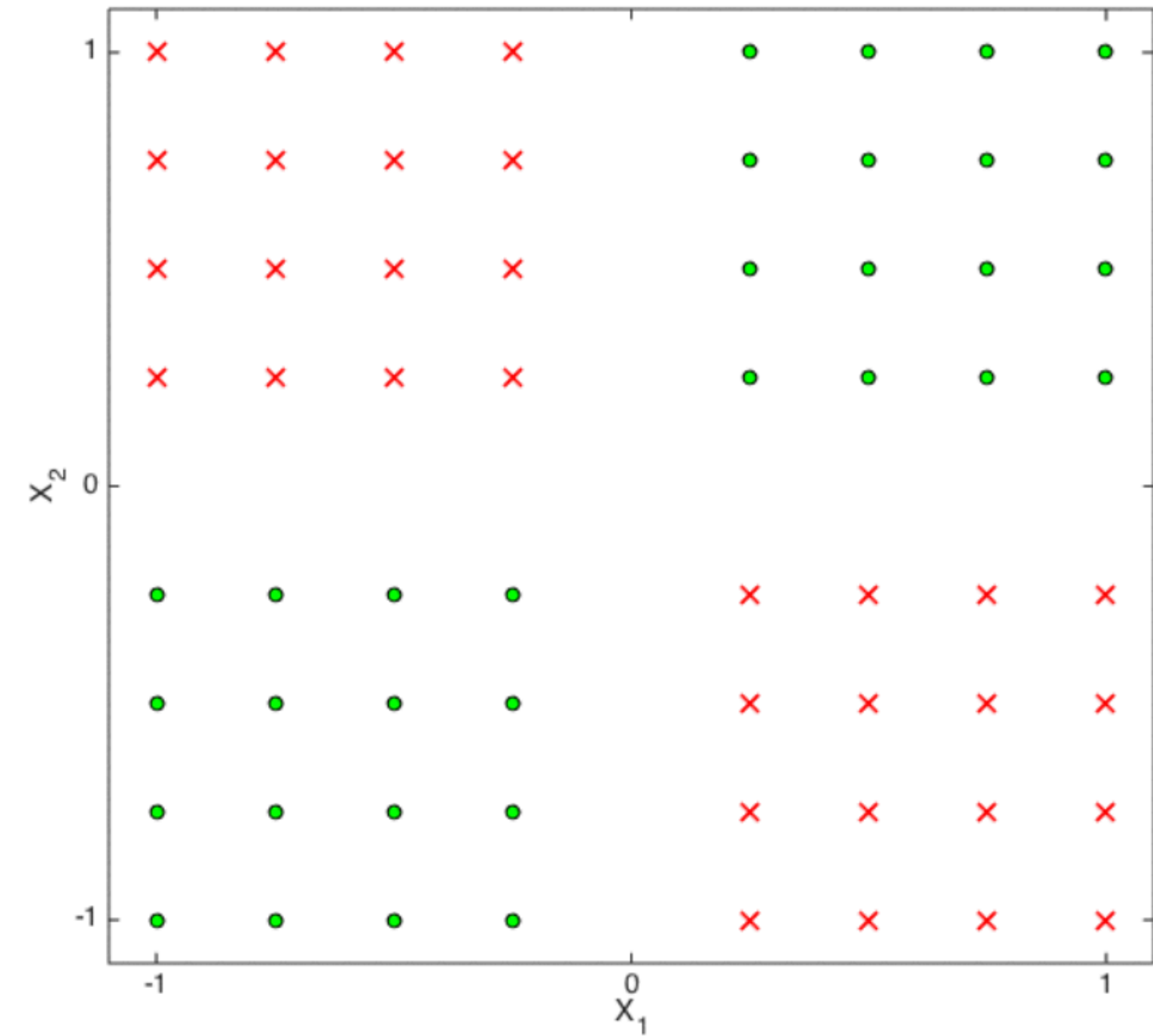
# Support Vector Classifier

We are still using a *linear* decision boundary.



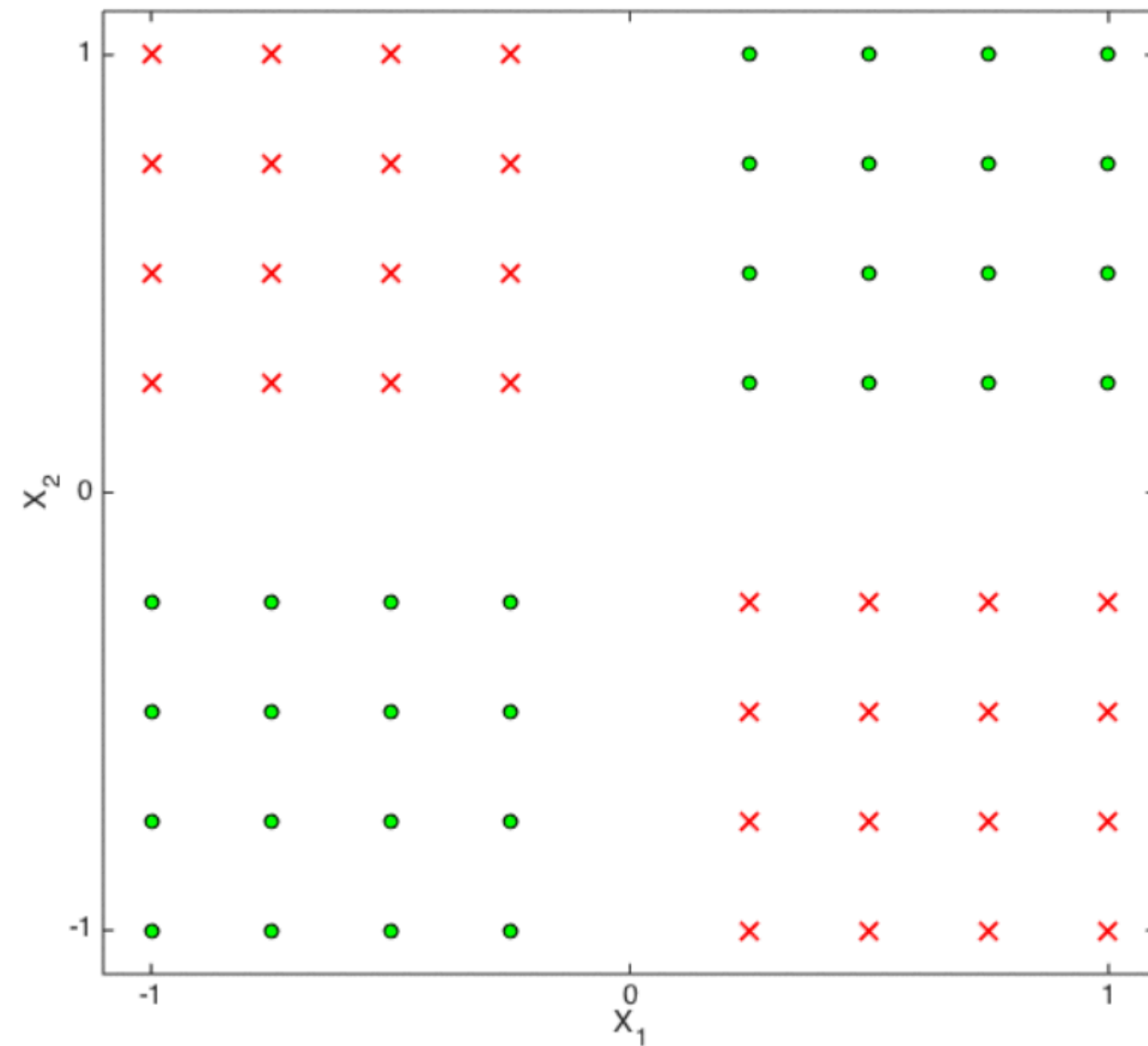
# Expanding Feature Space

Some datasets are not linearly separable, but they *become* linearly separable when transformed into a *higher* dimensional space.



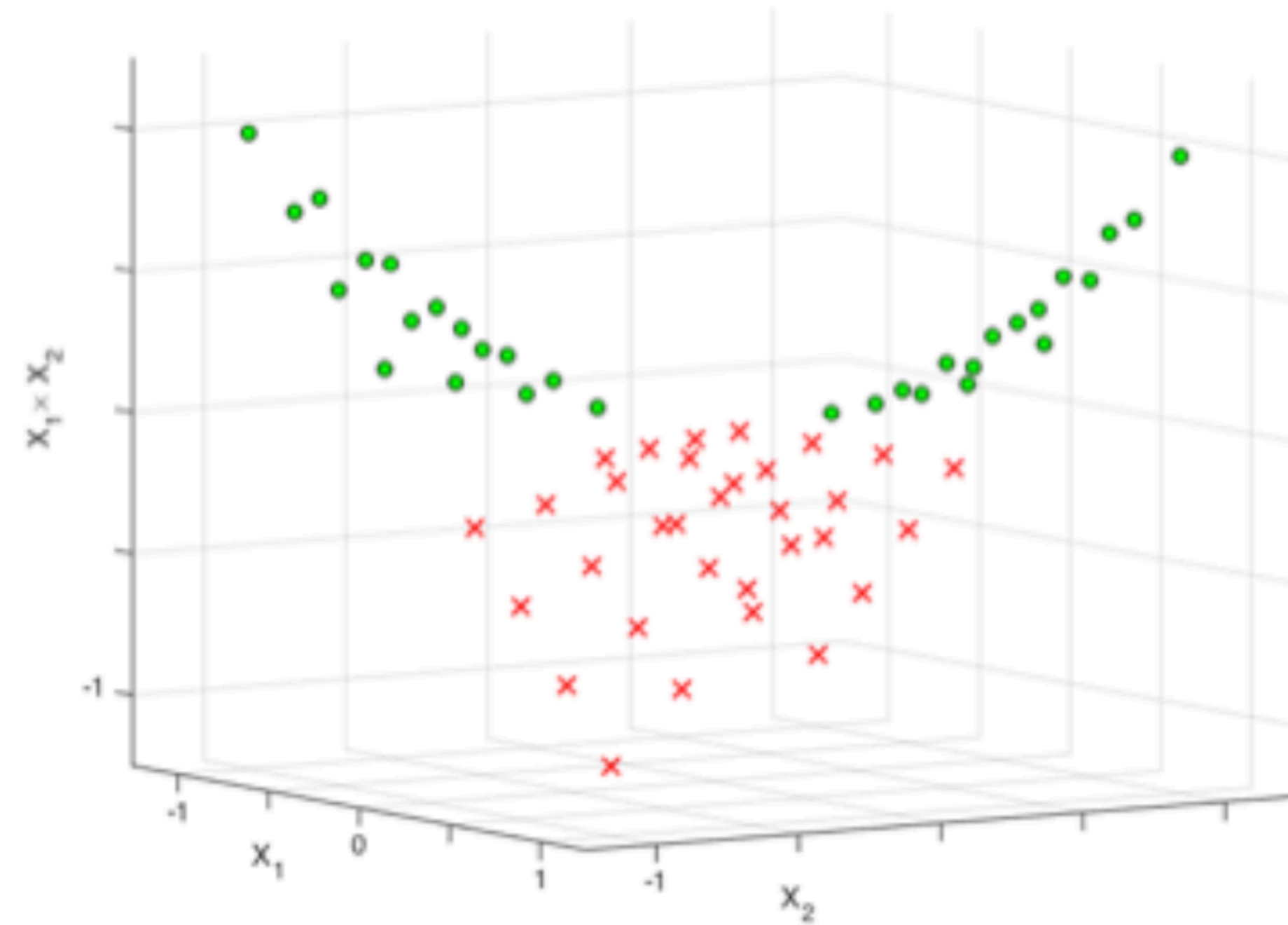
# Expanding Feature Space

**Original feature space**



variables  $x_1, x_2$

**New feature space**



variables  $x_1, x_2, x_1x_2$

# Expanding Feature Space

Recall that in linear regression, we created new features to capture non-linearity of data.

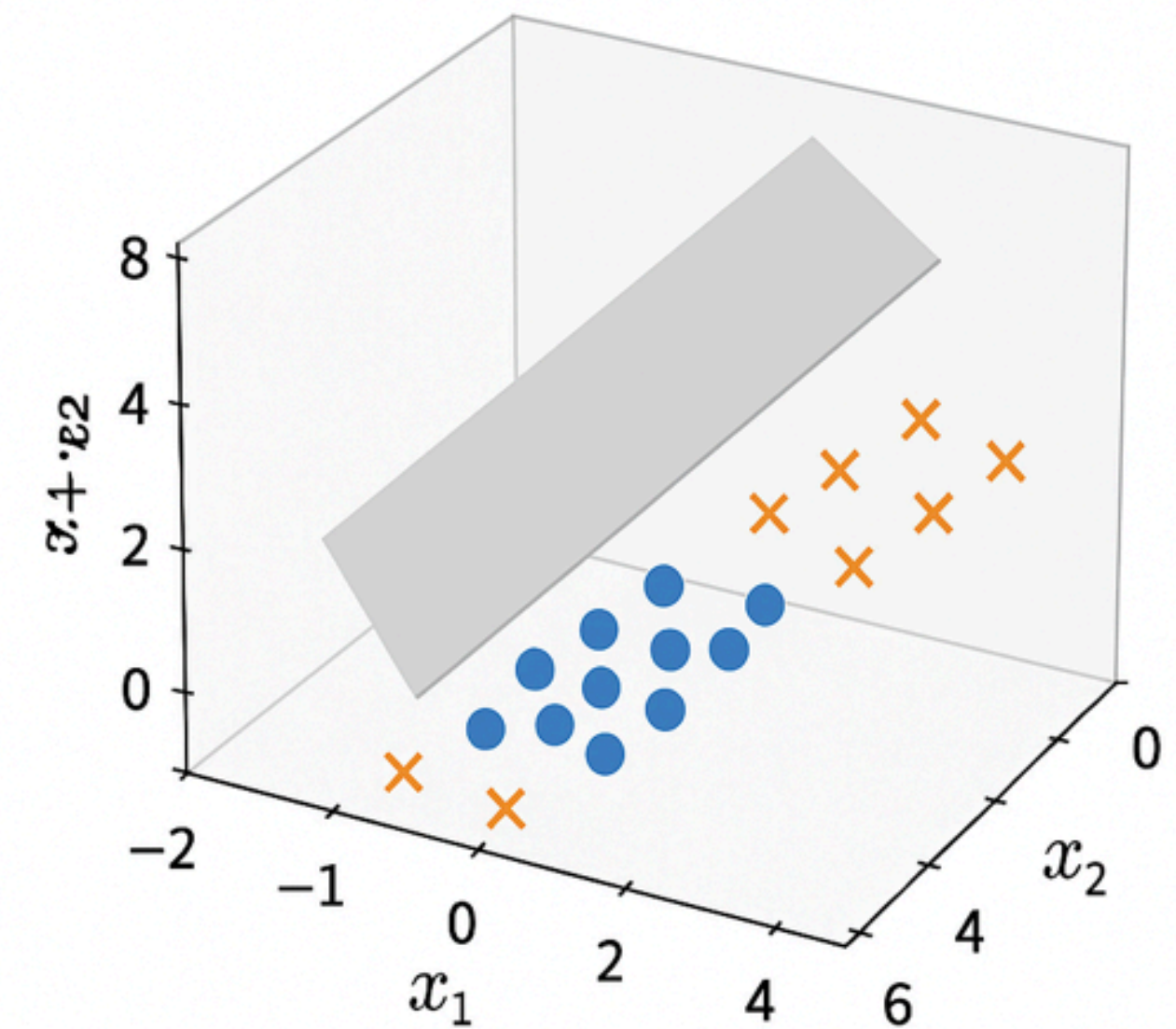
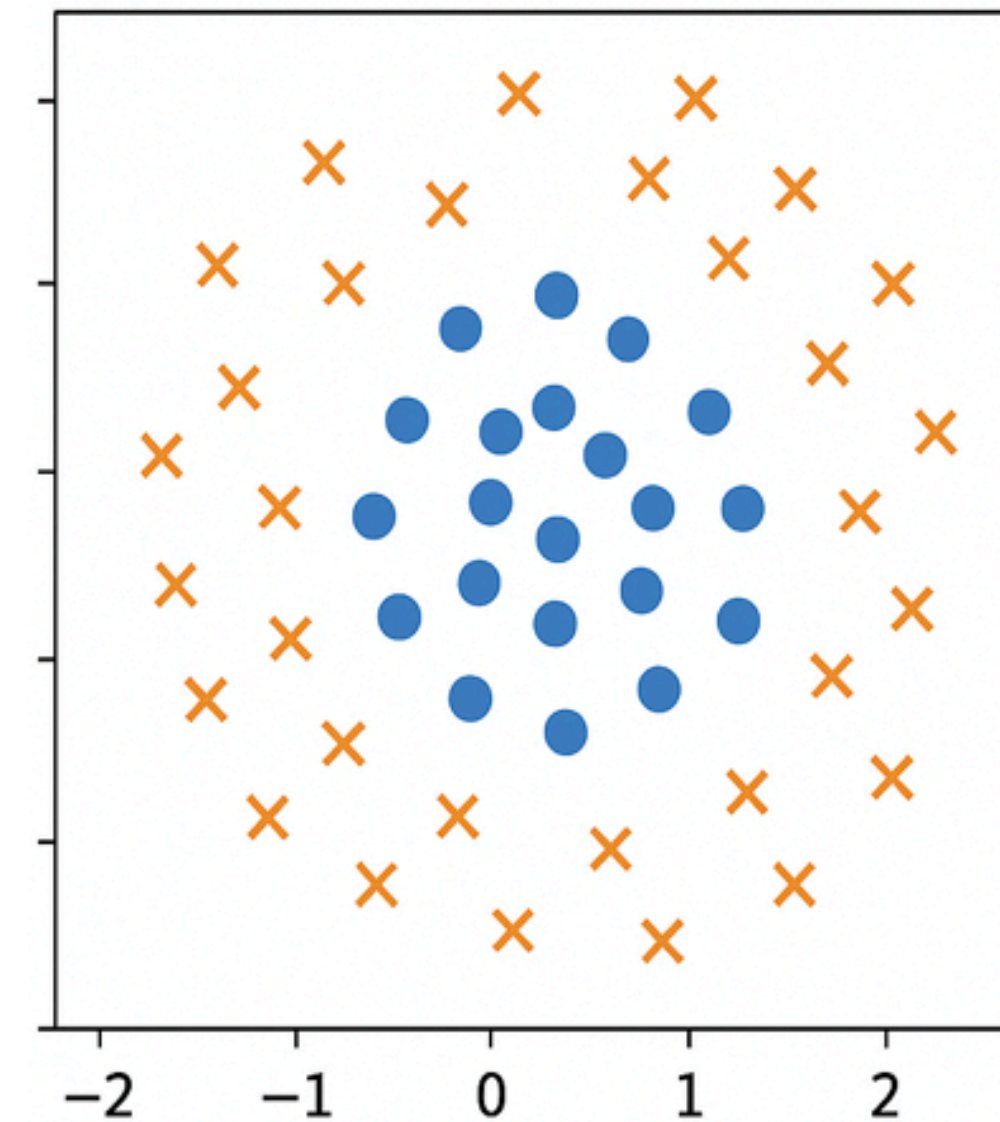
$$\vec{X} = (X_1, X_2, \dots, X_p)$$

We can expand the feature space to include e.g.  $2p$  features.

$$\begin{aligned}\vec{\tilde{X}} &= (X_1, X_1^2, X_2, X_2^2, \dots, X_p, X_p^2) \\ &= (\tilde{X}_1, \tilde{X}_2, \tilde{X}_3, \tilde{X}_4, \dots, \tilde{X}_{2p-1}, \tilde{X}_{2p})\end{aligned}$$

# Expanding Feature Space

- If our data isn't linearly separable in the original space, we can **add new features** (like polynomial terms) to make it separable.
- **Example:** In 2D, data that looks like a circle can't be split with a straight line.



- But if we add a feature like  $x_1^2 + x_2^2$ , then the boundary becomes linear in this higher-dimensional space.
- In the original space, the decision boundary looks **non-linear** (like an ellipse), but in the expanded space, it's just a hyperplane.

# Expanding Feature Space

Support vector classifier will find a hyperplane in  $2p$  dimensions.

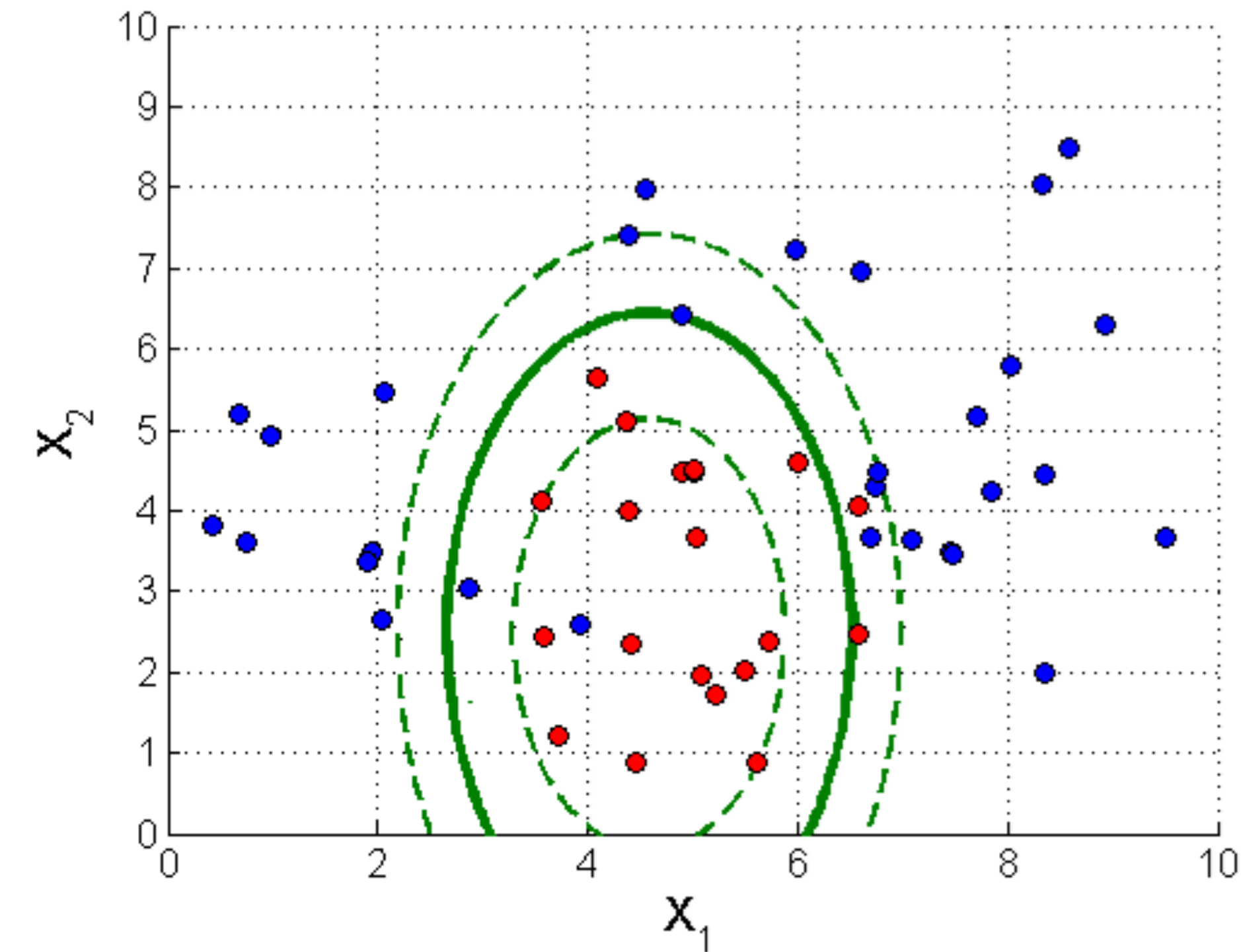
$$\beta_0 + \beta_1 \tilde{X}_1 + \beta_2 \tilde{X}_2 + \dots + \beta_{2p-1} \tilde{X}_{2p-1} + \beta_{2p} \tilde{X}_{2p} = 0$$

Hyperplane will be non-linear in *original* feature space.

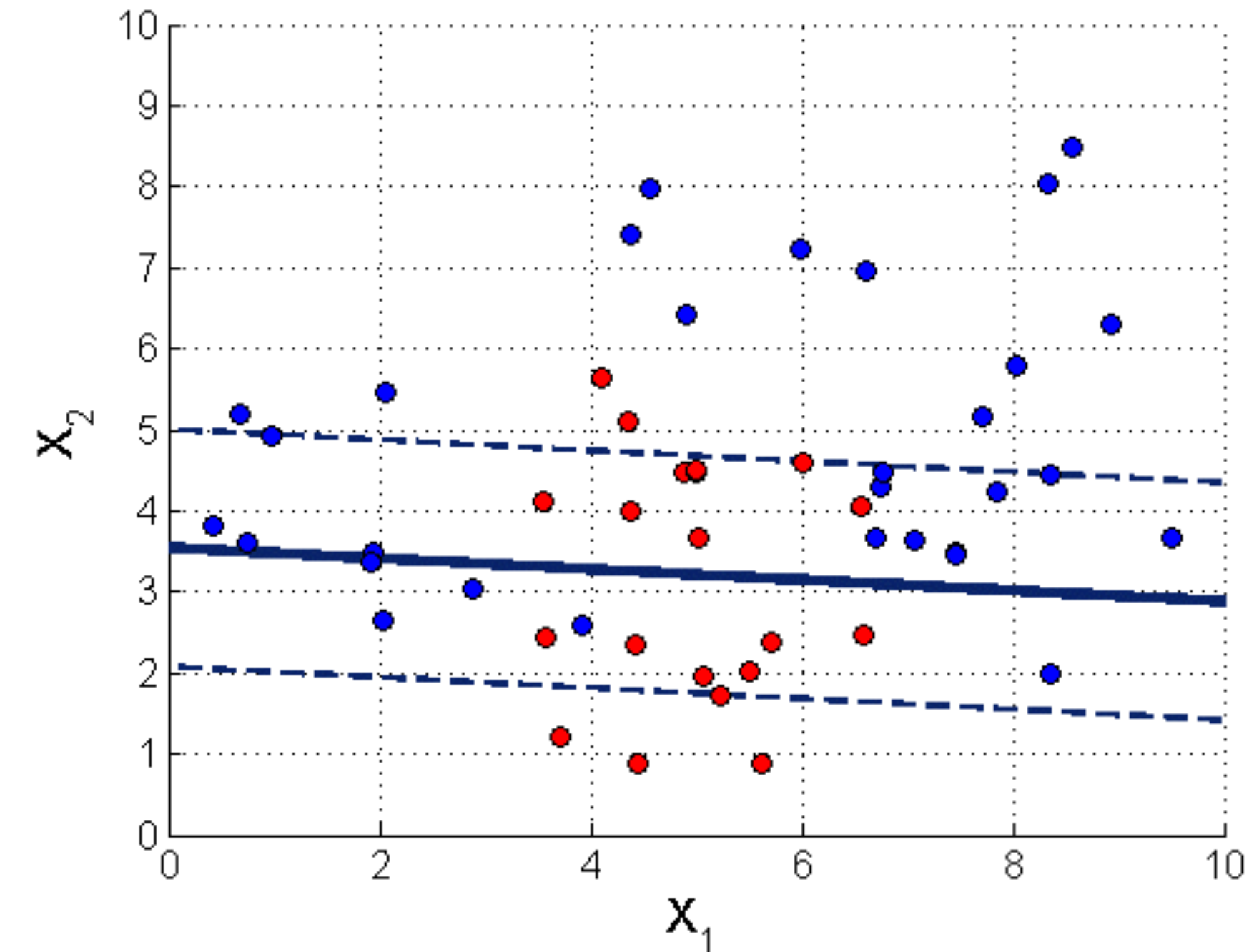
In this case, it is an ellipse.

$$\beta_0 + \beta_1 X_1 + \beta_2 X_1^2 + \dots + \beta_{2p-1} X_p + \beta_{2p} X_p^2 = 0$$

# Non-linear Decision Boundary



By expanding the feature space, we can get **curved boundaries** in the original feature space.



**Example:** Instead of a straight line, the boundary might now be a circle, an ellipse, or another curve that better separates the classes.

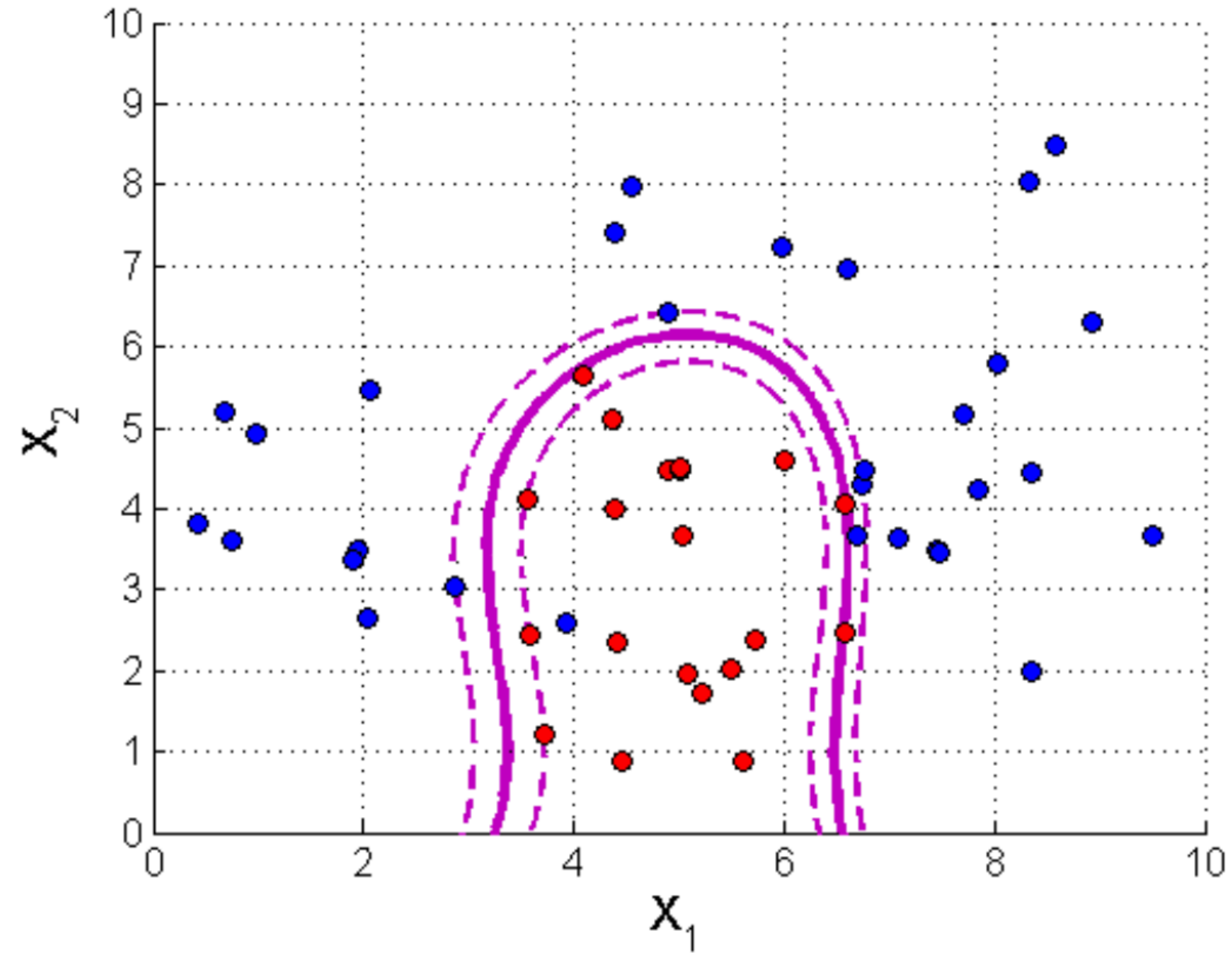
This shows the **power of SVMs**: they don't have to be stuck with straight lines if we transform the data first.

# The Problem With Expanding

Expanding features sounds great — just add higher-order terms, products, etc.

But in practice:

- Adding too many features can make the model **overfit** (too complex).
- It also becomes **computationally expensive** — working in very high dimensions is slow and hard.



# The Problem With Expanding

So we need a **smarter way** to get the benefit of high-dimensional features **without explicitly computing them**.

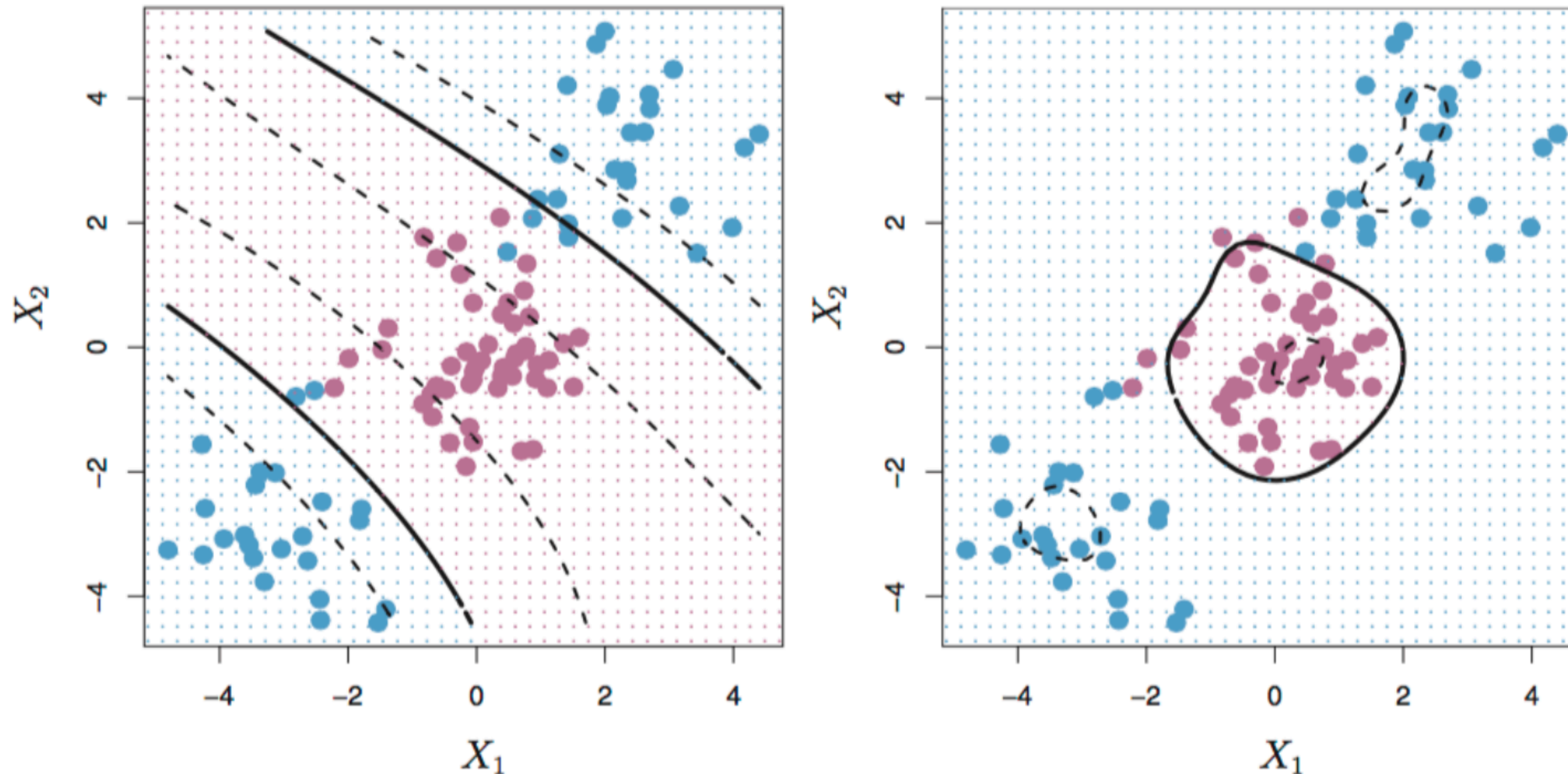
This sets up the idea of **kernels**:

kernels let us work in a high-dimensional space efficiently, without ever calculating the expanded features directly.

# Support Vector Machines

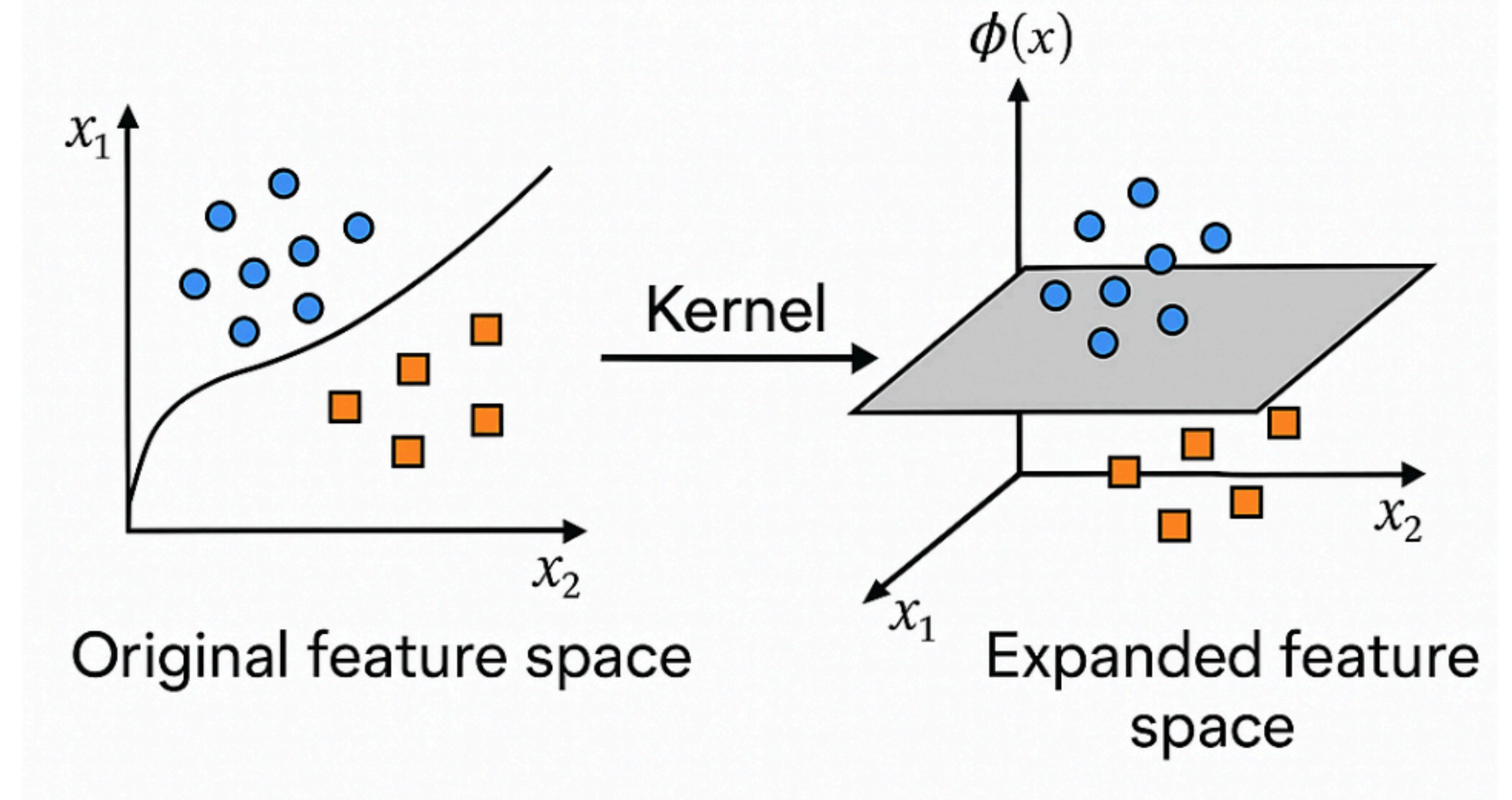
# Support Vector Machines (SVM)

Extends the support vector classifier by using **kernel functions** to achieve non-linear decision boundaries.



# Support Vector Machines (SVM)

**Kernel function:** generalization of inner product. It takes in two arguments and *implicitly* computes their inner product in some feature space.



A **kernel function** is a way to measure similarity between two data points.

- It generalizes the **inner product** (dot product).
- It lets us **implicitly map data into a higher-dimensional space** without actually computing all those extra features.
- This makes it possible for SVMs to find **non-linear decision boundaries** efficiently.

# Linear SVM

**Linear SVM:** A support vector classifier can be represented as

$$f(x) = \beta_0 + \sum_{i \in S} \alpha_i \langle x, x_i \rangle$$

# Linear SVM

**Linear SVM:** A support vector classifier can be represented as

$$f(x) = \beta_0 + \sum_{i \in S} \alpha_i \langle x, x_i \rangle \quad \text{inner product}$$

recall:  $f(x) > 0$  is one class  
 $f(x) < 0$  is another

$S$  is the set of  
support vectors

where

$$\langle \vec{u}, \vec{v} \rangle = \sum_{j=1}^p u_j v_j$$

# General SVM

In general a support vector classifier can be represented as

$$f(x) = \beta_0 + \sum_{i \in S} \alpha_i \langle x, x_i \rangle$$

In SVM we replace the inner product with some kernel function

$$f(x) = \beta_0 + \sum_{i \in S} \alpha_i K \left( x^{(i)}, x \right)$$

# Properties of Kernels

## 1. Generalization of inner product:

- A kernel is like a “fancy dot product.”
- Instead of multiplying features directly, it measures similarity in a possibly higher-dimensional space.

$$\phi : \mathcal{X} \rightarrow \mathcal{X}^\phi$$

$$x \mapsto \phi(x)$$

$$K(x, x') = \langle \phi(x), \phi(x') \rangle_{\mathcal{X}^\phi}$$

**2. Symmetry:** Doesn't matter the order — similarity from A to B is the same as B to A.

$$K(x, x') = K(x', x)$$

**3. Similarity Measure:** Kernels tell us how similar two points are:

- If two points are close  $\rightarrow$  kernel value is large.
- If they are far apart  $\rightarrow$  kernel value is small.

# Common SVM Kernels

- Linear kernel

$$K(x, x') = \langle x, x' \rangle$$

- Polynomial kernel (degree  $p$ )

$$K(x, x') = (1 + \langle x, x' \rangle)^p$$

- Radial basis kernel

$$K(x, x') = \exp(-\gamma ||x - x'||^2)$$

# SVM Summary

## Pros:

- Regularization parameter  $C$  helps avoid overfitting
- Use of kernel gives flexibility in shape of decision boundary
- Optimization problem is convex — unique solution

## Cons:

- Must tune hyper parameters (e.g.  $C$ , kernel function)
- Must formulate as binary classification
- Difficult to interpret

# SVM With More Than 2 Classes

SVMs are designed for binary classification, given the nature of a separating hyperplane.

We can adapt SVMs to perform classification when we have more than 2 classes.

Popular approaches:

- One-versus-one
- One-versus-all

# One -versus-one Classification

Construct an SVM for each pair of classes.

For  $k$  classes, this requires training  $k(k-1)/2$  SVMs.

To classify a new observation, apply all  $k(k-1)/2$  SVMs to the observation.

Take the most frequent class among pairwise results as the predicted class.

**Con:** computationally expensive for large  $k$ .

# One -versus-all Classification

Construct an SVM for each class against the  $k-1$  other classes pooled together.

For  $k$  classes, this requires training  $k$  SVMs.

Distance to separating hyperplane is a proxy for confidence of classification. For new observation, choose the “highest confidence” class as the prediction.

**Con:** may exacerbate class imbalances, distance to hyperplane may not correspond well to confidence.



**15 minutes break**

# Why Regularization?

# Bias-Variance Tradeoff

## Introduction

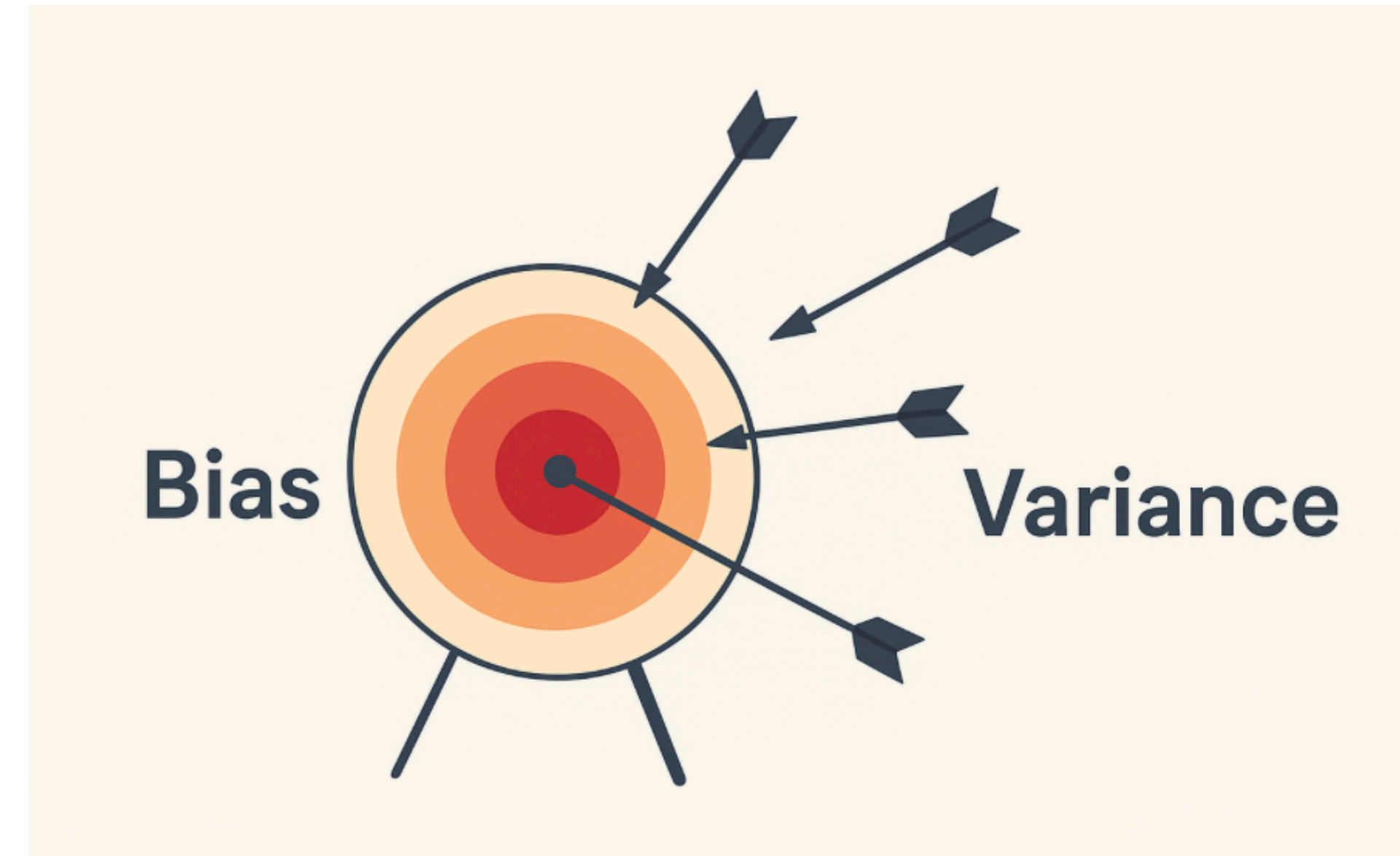
- In supervised learning, the error in predictions can be decomposed into **Bias**, **Variance**, and **Irreducible Error** (noise).
- Understanding this decomposition helps us balance underfitting and overfitting.
- Error Decomposition Formula:

$$\text{Expected Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

# Bias-Variance Tradeoff

## Introduction

- **Bias**
  - Error from incorrect assumptions in the model.
  - High bias → model is too simple (e.g., linear model for nonlinear data).
  - Leads to underfitting.
- **Variance**
  - Error from sensitivity to training data fluctuations.
  - High variance → model is too complex (captures noise).
  - Leads to overfitting.
- **Irreducible Error (Noise):**
  - Error due to inherent randomness in the data.
  - Cannot be reduced no matter how good the model is.



# Bias-Variance Tradeoff

## Model Assessment

- There is a tradeoff between bias and variance.
- If our model is too “simple” and has very few parameters, then it may have large bias (but small variance), and it typically may suffer from underfitting.
- If it is too “complex” and has very many parameters, then it may suffer from large variance (but have smaller bias), and thus overfitting.

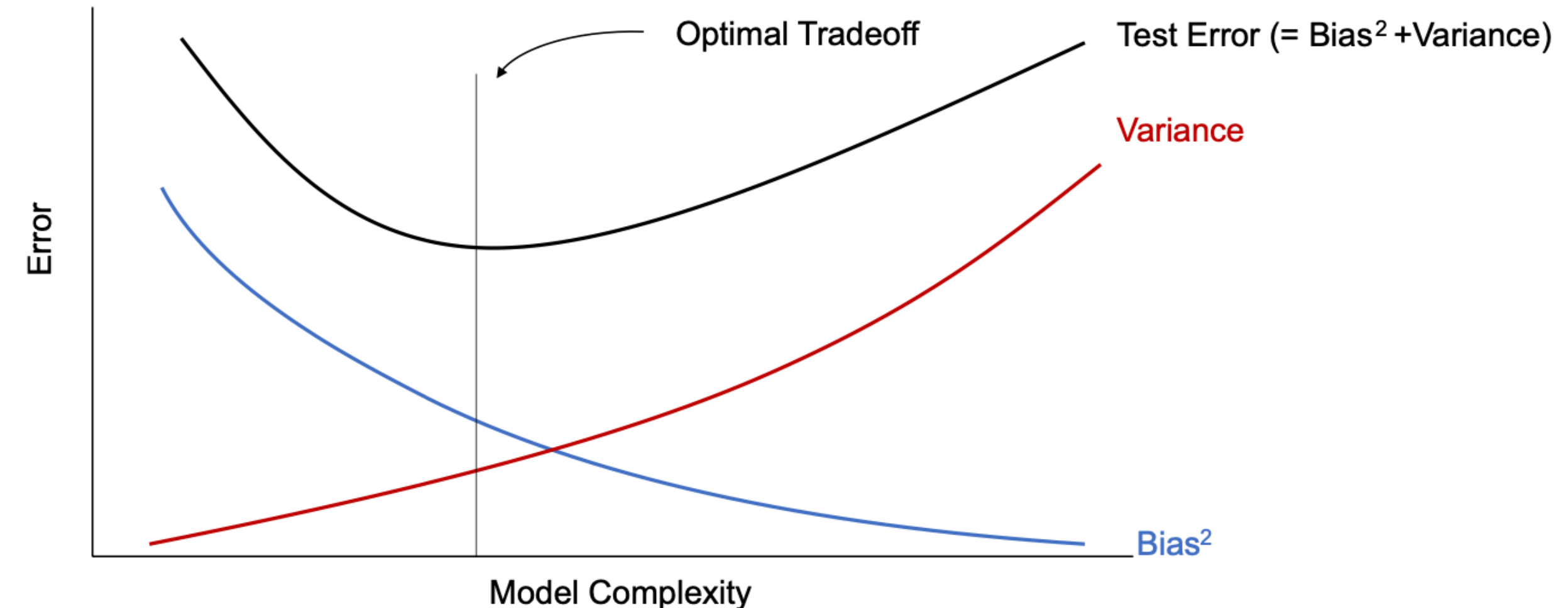
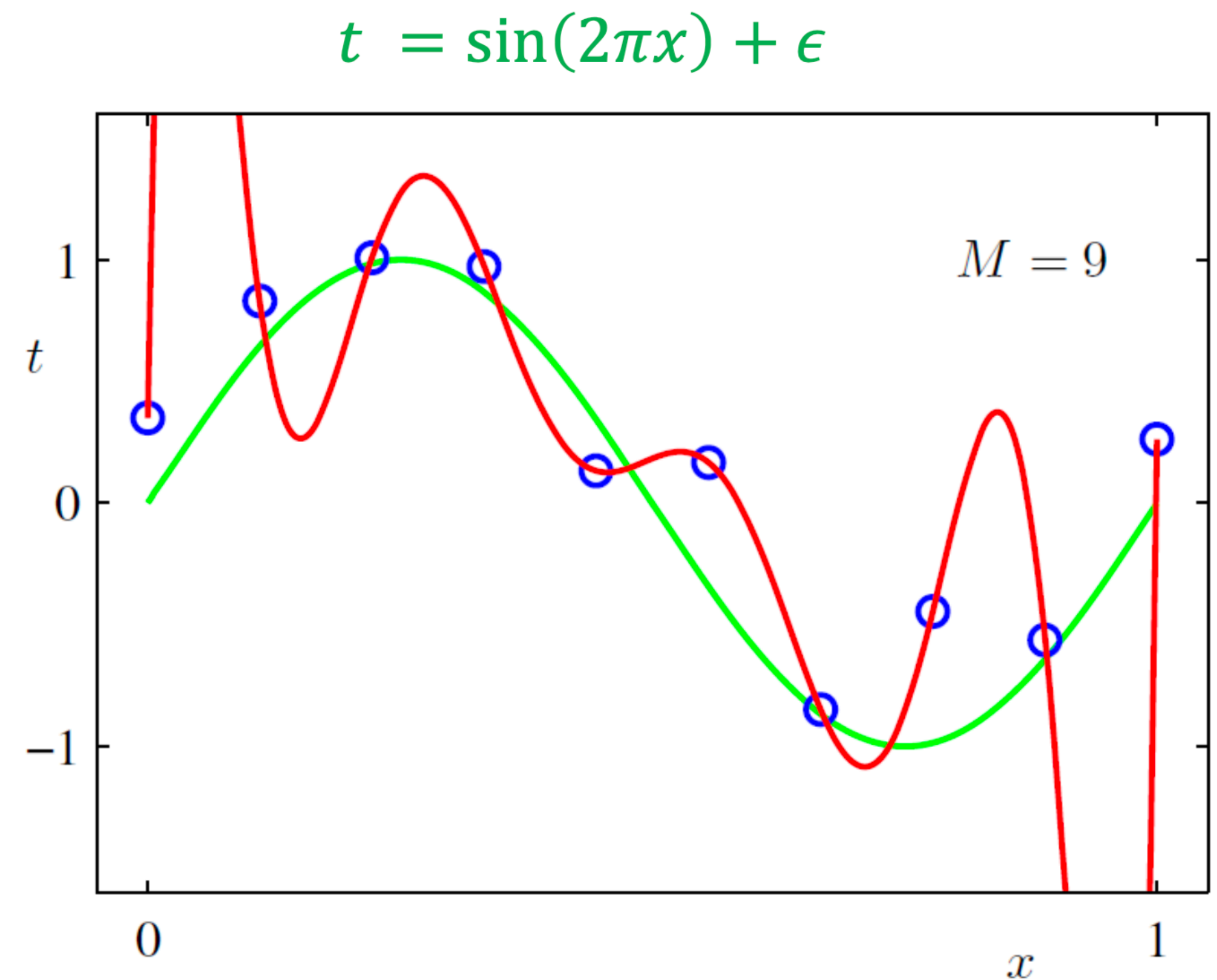


Figure 8.8: An illustration of the typical bias-variance tradeoff.

# Overfitting

# Overfitting

- Minimizing the training error is a reasonable thing to do, but it's possible to minimize it “too well”
- If your function matches the training data well but is not learning general rules that will work for new data, this is called **overfitting**



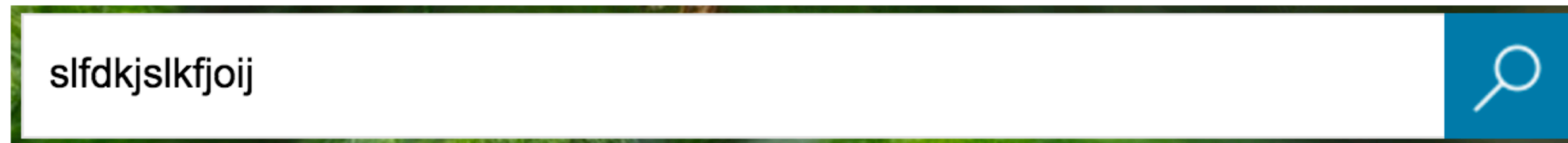
# Overfitting: Logistic Regression

Suppose you are a search engine and you build a classifier to infer whether a user is over the age of 65 based on what they've searched.



# Overfitting: Logistic Regression

One person in your dataset searched the following typo:

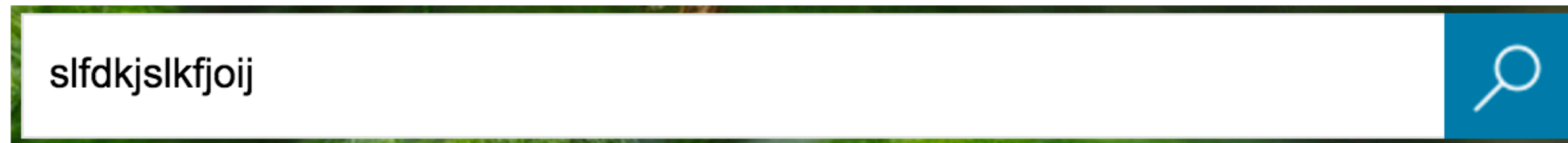


This person was over age 65.

Optimizing the logistic regression loss function, we would learn that anyone who searches *slfdkjslkfjoij* is over 65 with probability 1.

# Overfitting: Logistic Regression

One person in your dataset searched the following typo:

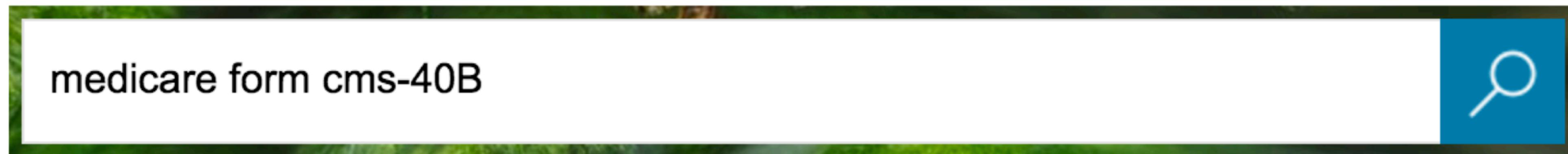


Hard to conclude much from 1 example.

Don't really want to classify all people who make this typo in the future this way.

# Overfitting: Logistic Regression

One person in your dataset searched the following:

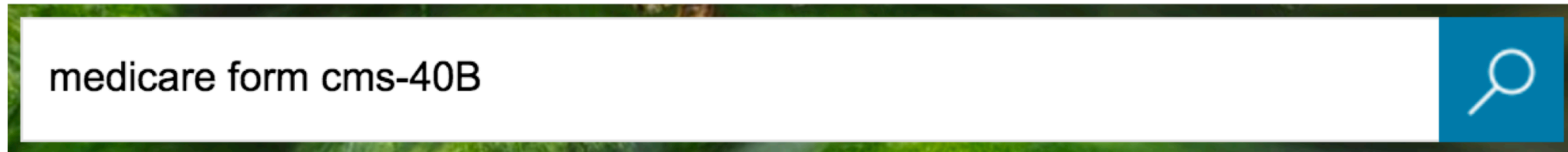


All ten people were over age 65.

Optimizing the logistic regression loss function, we would learn that anyone who searches this query is over 65 with probability 1.

# Overfitting: Logistic Regression

One person in your dataset searched the following:



This query is probably good evidence that someone is older than (or near) 65.

Still: what if someone searched this who otherwise had hundreds of queries that suggested they were younger?

They would still be classified  $> 65$  with probability 1.

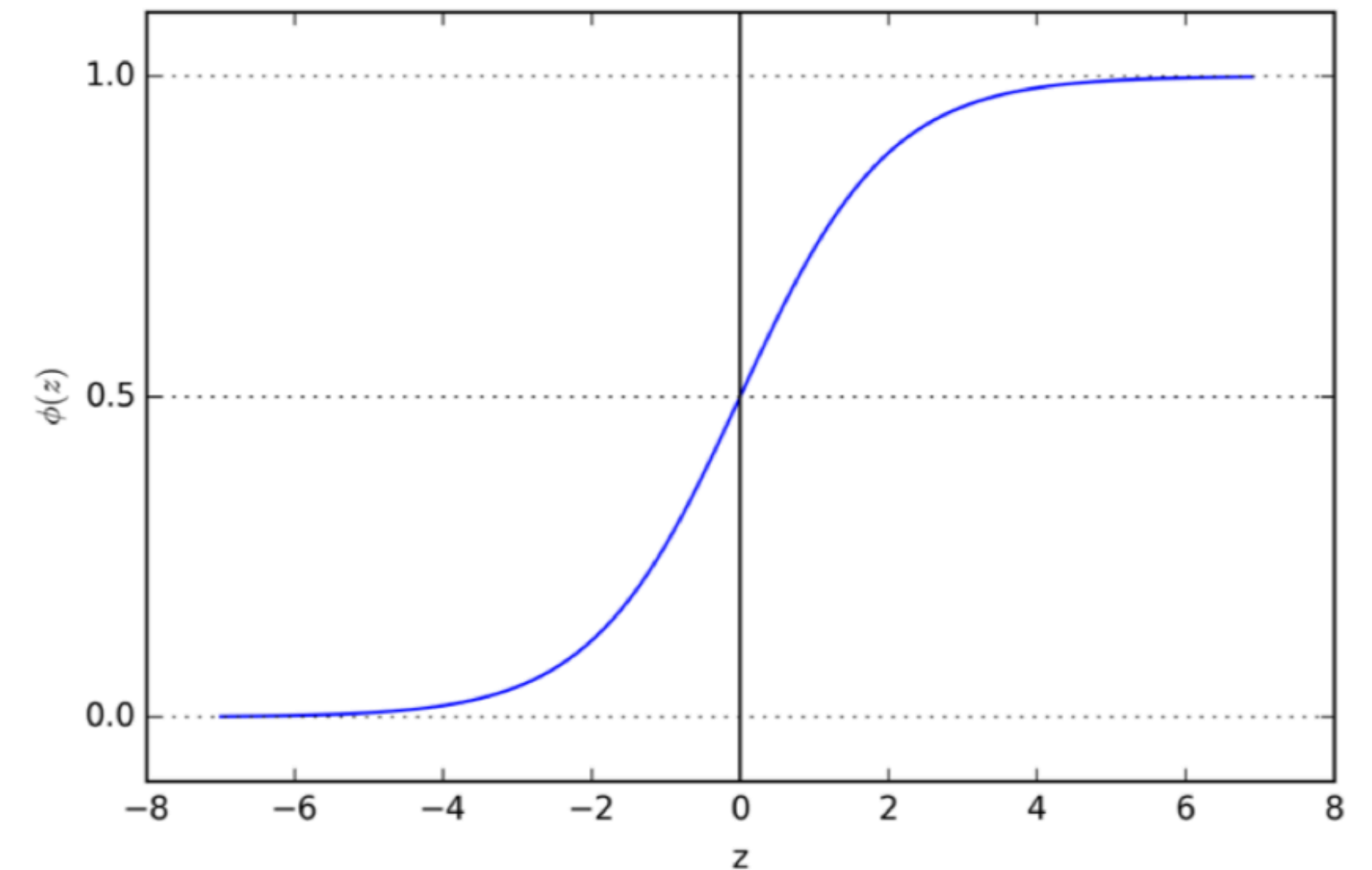
**The probability 1 overrides other features in logistic regression.**

# Overfitting: Logistic Regression

Risk of **numeric instability** if weights get too large.

Recall the logistic function:

$$\phi(z) = \frac{1}{1 + e^{-z}}$$



This To make the model predict **exact probability 1 (or 0)**, the logistic function requires  $z \rightarrow +\infty$  (for 1) or  $z \rightarrow -\infty$  (for 0).

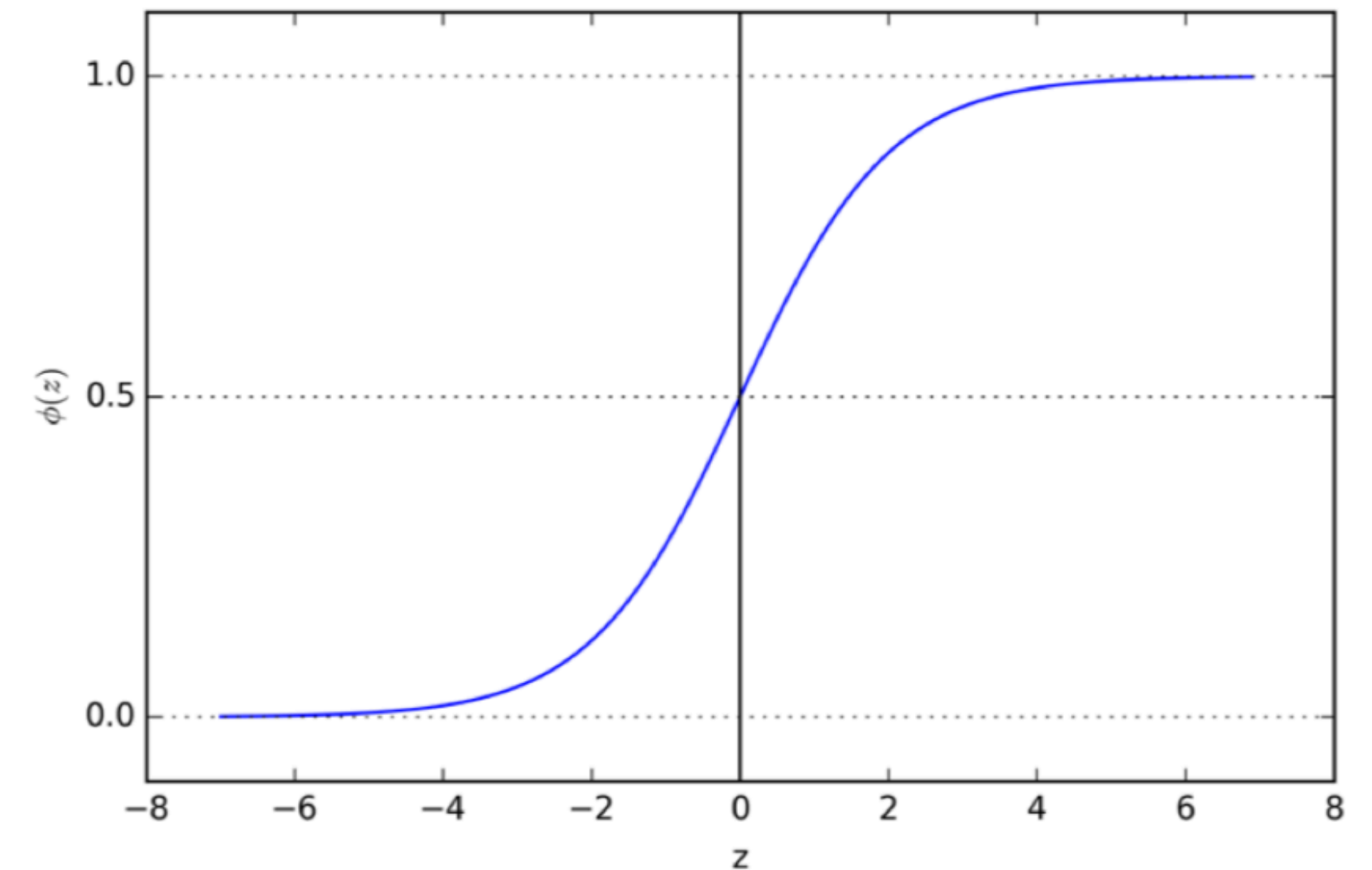
That means the **weights w** must grow extremely large in magnitude.

# Overfitting: Logistic Regression

Risk of **numeric instability** if weights get too large.

Recall the logistic function:

$$\phi(z) = \frac{1}{1 + e^{-z}}$$



To make the model predict **exact probability 1 (or 0)**, the logistic function requires  $z \rightarrow +\infty$  (for 1) or  $z \rightarrow -\infty$  (for 0).

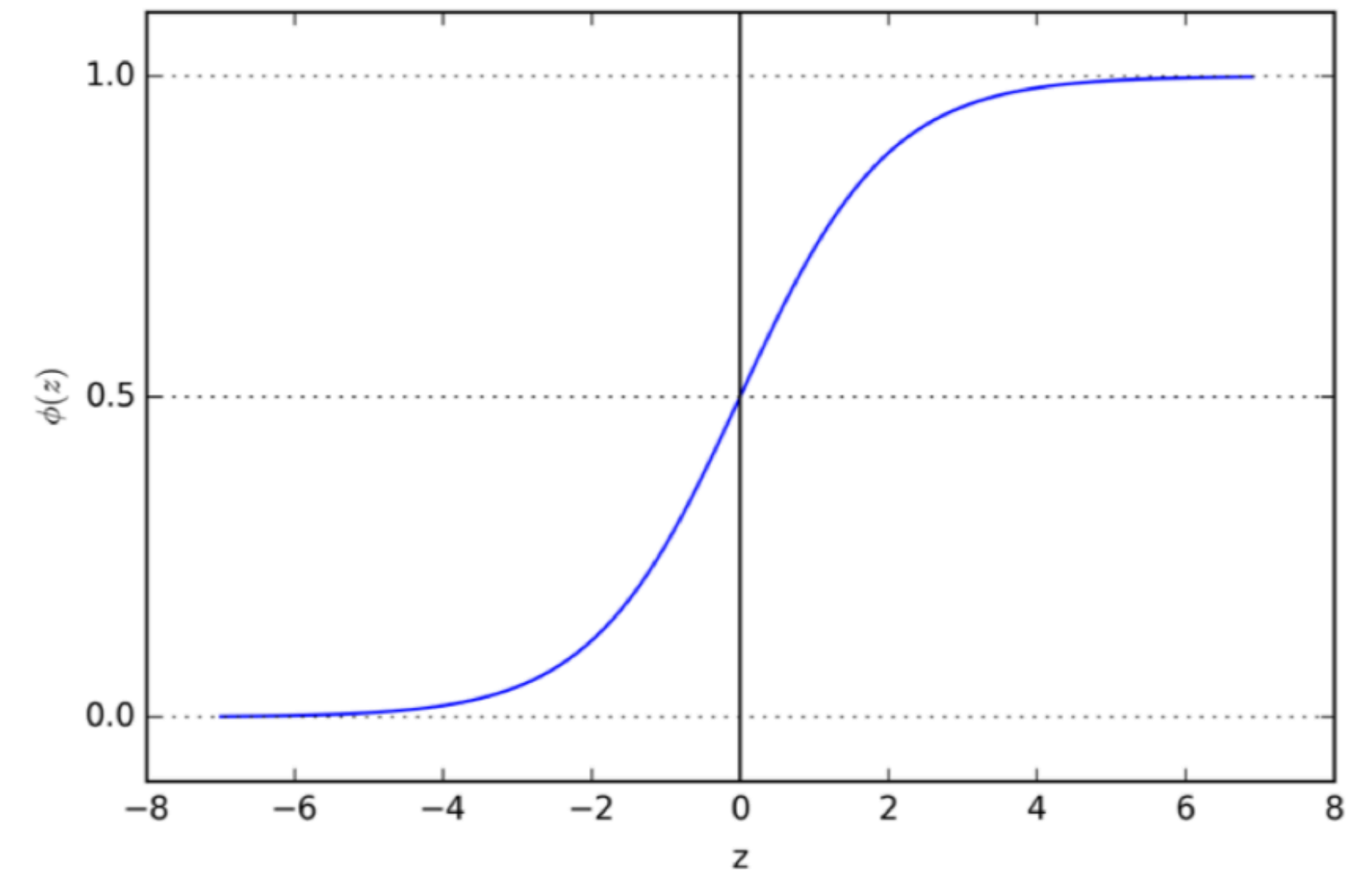
That means the **weights w** must grow extremely large in magnitude.

# Overfitting: Logistic Regression

Risk of numeric instability if weights get too large.

Recall the logistic function:

$$\phi(z) = \frac{1}{1 + e^{-z}}$$



This leads to **numerical instability**:

- Computation may overflow (become infinity) or underflow (become 0).
- Gradient descent updates can get stuck because the loss landscape flattens when weights are huge.
- The model wastes computation trying to chase “probability 1.000000...” instead of being satisfied with, say, 0.99.

# Overfitting: Logistic Regression

Example:

$$\phi(z) = \frac{1}{1 + e^{-z}}$$

Suppose logistic regression predicts:

- ◆ If  $z=5$ :  $\phi(5) \approx 0.993$ .
- ◆ If  $z=10$ :  $\phi(10) \approx 0.99995$ .
- ◆ If  $z=100$ :  $\phi(100) \approx 1.0$  (to machine precision).

Notice how going from  $5 \rightarrow 10$  already pushes probability extremely close to 1. Beyond that, pushing  $z$  to 100 just causes overflow in computing  $e^{-z}$ .

# Why Regularization?

So:

without **regularization**, logistic regression might keep increasing weights unnecessarily to try to make probabilities exactly 0 or 1 — which is both **numerically unstable** and **bad for generalization**.

# Classic Regularization Techniques

# Regularization Definition

- **Regularization** refers to the act of modifying a learning algorithm to favor “simpler” prediction rules to avoid overfitting.
- Most commonly, regularization refers to modifying the loss function to **penalize** certain values of the weights you are learning.
- Specifically, penalize weights that are large.

# Regularization Definition

- **How do we define** whether weights are *large*?

$$d(\mathbf{w}, \mathbf{0}) = \sqrt{\sum_{i=1}^k (w_i)^2} = \|\mathbf{w}\|$$

- This is called the **L2 norm** of **w**.
- A norm is a measure of a vector's length
- Also called the Euclidean norm

# L2 Regularization Method

Let's consider a simple logistic regression loss function:  $L(W)$

We add a penalty to it:  $L(w) + \lambda ||w||^2$

Here  $||w||^2 = \sum_i w_i^2$  is the squared length of the weight vector.

$\lambda$  is a hyperparameter that controls how strong the penalty is.

# L2 Regularization Method

Why squared?

$$\|w\|^2 = \sum_i w_i^2$$

Squared form is easier to work with mathematically (no square roots in derivatives).

It's still a measure of how big the weights are.

# L2 Regularization Method

## Intuition

$$L(w) + \lambda \|w\|^2$$

Minimizing this new loss function means the model must **balance two goals**:

1. Fit the training data (make  $\mathcal{L}(w)$  small).
2. Keep the weights small (make  $\|w\|^2$  small).

So the model won't give extreme importance to a single feature unless it's really useful.

# L2 Regularization Method

## Computational Benefits

$$L(w) + \lambda ||w||^2$$

- Without regularization, weights can blow up to huge values (like we saw in slide 18).
- With L2 regularization, the penalty prevents weights from growing too much.
- This stabilizes gradient descent and avoids overflow problems.

# L2 Regularization Method

## Generalization Benefits

$$L(w) + \lambda ||w||^2$$

- By discouraging large weights, the model doesn't overfit to rare quirks in training data.
- It only gives large weights to features that have **strong, consistent evidence**.
- This improves test performance because the model learns more “general” rules.

# Regularization Method

## General Form

$$L(\mathbf{w}) + \lambda \underbrace{R(\mathbf{w})}$$

- This is called the regularization term or **regularizer** or **penalty**
- The squared L2 norm is one kind of penalty, but here are others
- $\lambda$  is called the regularization **strength**

# Regularization Method

## Ridge Regularization

$$L(w) + \lambda \|w\|^2$$

- When the regularizer is the squared L2 norm  $\|\mathbf{w}\|^2$ , this is called L2 regularization.
- This is the most common type of regularization
- When used with linear regression, this is called *Ridge regression*.
- Logistic regression implementations usually use L2 regularization by default.
- L2 regularization can be added to other algorithms like perceptron (or any gradient descent algorithm)

# Ridge Regularization Method

## Logistic Regression

- Logistic regression without regularization:
  - We want to predict  $y \in \{0,1\}$  from input  $x$ .

- Model:

$$p(y = 1 \mid x; w) = \sigma(w^T x) = \frac{1}{1 + e^{-w^T x}}$$

- Loss Function:

$$L(w) = - \sum_{i=1}^n [y_i \log \sigma(w^T x_i) + (1 - y_i) \log(1 - \sigma(w^T x_i))]$$

# Ridge Regularization Method

## Logistic Regression

$$p(y = 1 \mid x; w) = \sigma(w^T x) = \frac{1}{1 + e^{-w^T x}}$$

- Logistic regression without regularization:

$$L(w) = - \sum_{i=1}^n [y_i \log \sigma(w^T x_i) + (1 - y_i) \log(1 - \sigma(w^T x_i))]$$

- Logistic Regression with Ridge regularization:

$$L_{reg}(w) = - \sum_{i=1}^n [y_i \log \sigma(w^T x_i) + (1 - y_i) \log(1 - \sigma(w^T x_i))] + \underbrace{\lambda \|w\|^2}$$

$$\text{Where: } \|w\|^2 = \sum_{j=1}^d w_j^2$$

and  $\lambda > 0$  is the regularization strength.

# Ridge Regularization Method

## Logistic Regression

- Logistic Regression with Ridge regularization:

$$L_{reg}(w) = - \sum_{i=1}^n [y_i \log(\frac{1}{1 + e^{-w^T x_i}}) + (1 - y_i) \log(1 - \frac{1}{1 + e^{-w^T x_i}})] + \lambda \sum_{j=1}^d w_j^2$$

Here,

- $n$  = number of training examples (the size of your dataset).
- $d$  = number of features (the dimension of each input vector  $x$ ).

So:

- The **loss term** (first big summation) goes over **all data points** → sum from  $i=1$  to  $n$ .
- The regularization term goes over all model parameters (weights) → sum from  $j=1$  to  $d$ .

# Ridge Regularization Method

## Summary

- Logistic Regression with Ridge regularization:

$$L_{reg}(w) = - \sum_{i=1}^n [y_i \log(\frac{1}{1 + e^{-w^T x_i}}) + (1 - y_i) \log(1 - \frac{1}{1 + e^{-w^T x_i}})] + \lambda \sum_{j=1}^d w_j^2$$

- The loss term (over n): fit the data.
- The **regularization term (over d)**: keep the model simple by controlling the size of weights.
- This balance is controlled by  $\lambda$ :
  - Large  $\lambda \rightarrow$  weights close to zero (risk of underfitting).
  - Small  $\lambda \rightarrow$  weights can grow big (risk of overfitting).

# Regularization Methods

## Choosing $\lambda$

$$L_{reg}(w) = - \sum_{i=1}^n [y_i \log(\frac{1}{1 + e^{-w^T x_i}}) + (1 - y_i) \log(1 - \frac{1}{1 + e^{-w^T x_i}})] + \lambda \sum_{j=1}^d w_j^2$$

- How to choose  $\lambda$ ?
- You'll play around with it in the homework, and we'll also return to this later in the semester when we discuss hyperparameter optimization.
- Other common names for  $\lambda$ :
  - *alpha* in **sklearn**
  - *C* in many algorithms
    - Usually *C* actually refers to the **inverse regularization strength,  $1/\lambda$**
  - Figure out which one your implementation is using (whether this will increase or decrease regularization)

# L1 Regularization Method

## Lasso Regularization

Another common regularizer is the L1 norm:

$$L_{reg}(w) = L(w) + \lambda \|w\|_1$$

Where,  $\|w\|_1 = \sum_{j=1}^d |w_j|$

When used with linear regression, this is called *Lasso Regularization*.

“*Lasso*” = Least Absolute Shrinkage and Selection Operator.

# L1 Regularization Method

## Lasso Regularization

$$L_{reg}(w) = L(w) + \lambda \|w\|_1$$

- Unlike L2 (which shrinks weights smoothly toward zero), **L1 actually pushes some weights to exactly zero.**
- This means L1 performs **feature selection** automatically:
  - Irrelevant features get weight = 0.
  - Important features keep a nonzero weight.
- Use L1 when you believe only a small number of features are truly important.

$$\|w\|_1 = \sum_{j=1}^d |w_j|$$

# L1 vs. L2 Regularization Method

| Feature            | L1 Regularization ( $\ w\ _1$ )                                           | L2Regularization ( $\ w\ _2^2$ )                                 |
|--------------------|---------------------------------------------------------------------------|------------------------------------------------------------------|
| Effect on weights  | Pushes many weights exactly to <b>0</b> (sparse model, feature selection) | Shrinks weights smoothly toward 0, but usually not exactly 0     |
| Geometry intuition | Diamond-shaped constraint → encourages axes (some weights zero)           | Circle-shaped constraint → spreads weights evenly                |
| Usecase            | Good when you expect only a few features matter (feature selection)       | Good when you think many features are useful but should be small |
| Optimization       | Harder (non-differentiable at 0)                                          | Easier (smooth derivatives)                                      |
| Name               | <b>Lasso</b> (Linear regression with L1)                                  | <b>Ridge</b> (Linear regression with L2)                         |

# Modern Regularization Techniques

# Elastic Net Regularization

L2 and L1 regularization can be combined

$$R(\mathbf{w}) = \lambda_2 \|\mathbf{w}\|^2 + \lambda_1 \|\mathbf{w}\|_1$$

$$L_{reg}(w) = L(w) + \lambda_1 \sum_{j=1}^d |w_j| + \lambda_2 \sum_{j=1}^d w_j^2$$

- Also called ***ElasticNet***
- Can work better than either type alone
- Encourages some weights to be zero but also keeps correlated features together.
- Can adjust hyperparameter to control which of the two penalties is more important

# Drop Out Regularization

- Randomly “drops” (sets to 0) a fraction of neurons during training.
- Prevents the network from relying too much on specific neurons.
- Encourages redundancy → each neuron must learn more robust features.
- At test time, all neurons are used (with outputs scaled).
- Think of it as **training many smaller networks inside the big one.**

# Data Augmentation Regularization

- Expands the training set by applying random transformations to inputs:
  - Images → flips, rotations, crops, color shifts.
  - Text → synonym replacement, back-translation.
  - Audio → noise injection, time shifting.
- Prevents overfitting by making the model robust to variations.
- It's like giving the model “extra experience” without collecting new data.

# Early Stopping Regularization

- During training, stop once validation error starts to increase (even if training error is still going down).
- Prevents the model from “memorizing” noise in the training data.
- A simple but very effective regularization method.
- Think of it as “**quit while you’re ahead**” in training.

# Batch Normalization (BatchNorm)

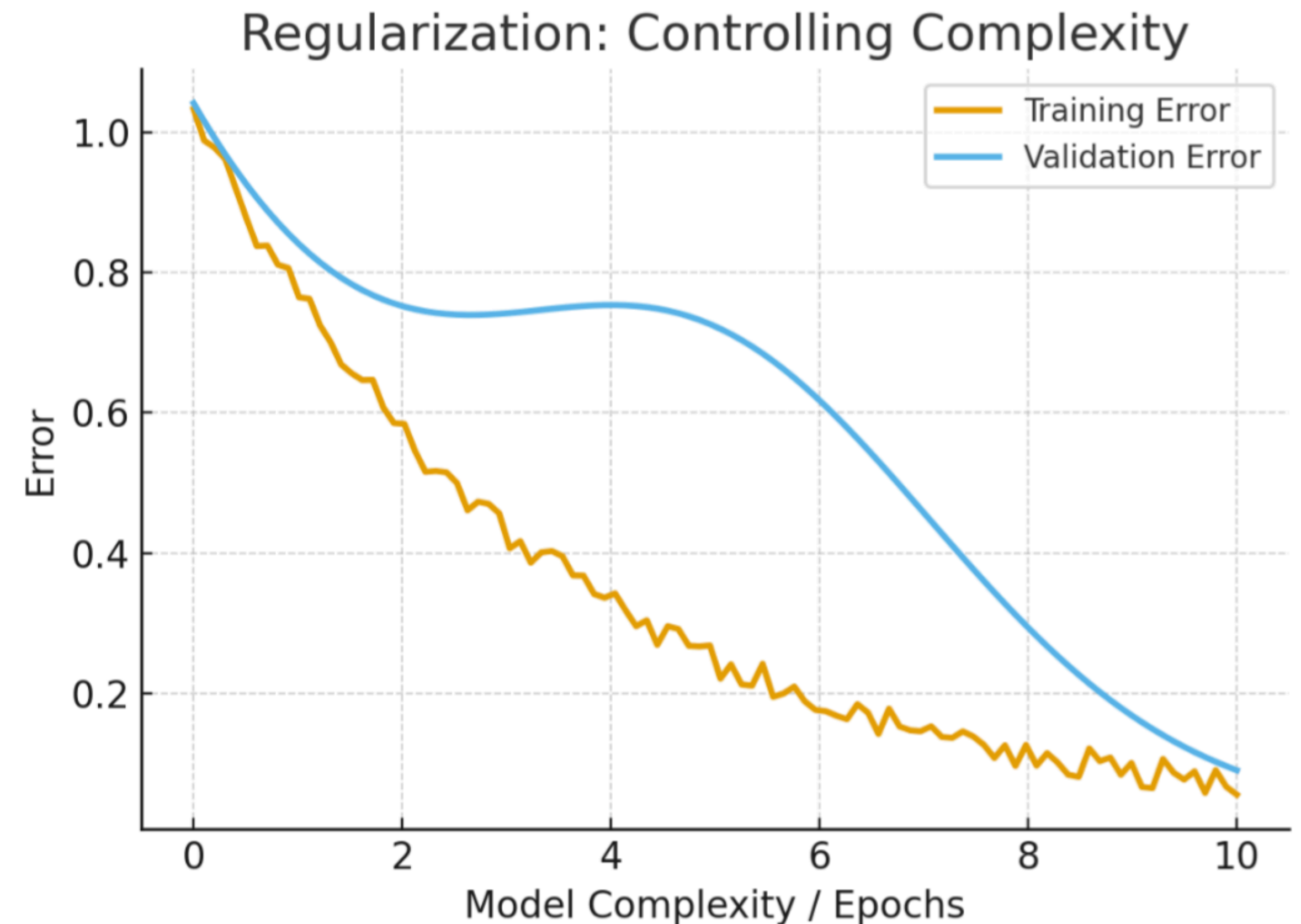
- Normalizes activations within each mini-batch (zero mean, unit variance).
- Acts like a form of regularization by reducing internal covariate shift.
- Helps optimization: smoother loss landscape, faster convergence.
- Often reduces need for other regularization (but not a full replacement).
- Think of it as **keeping the network's activations balanced** throughout training.

# Optimization Techniques

# From Regularization To Optimization

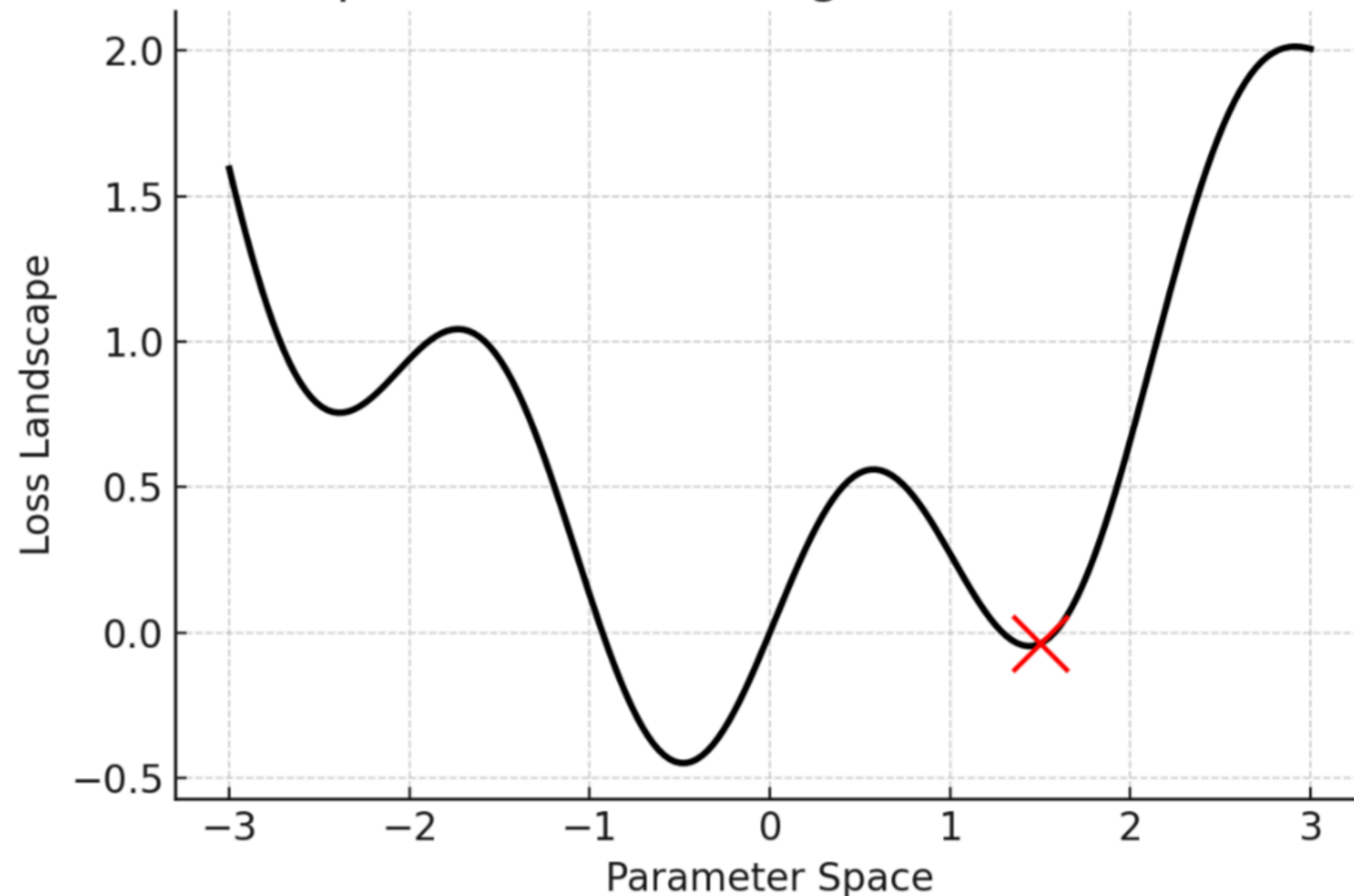
## Regularization

- Helps models **generalize better**
- Controls complexity (L1, L2, Elastic Net)
- Modern methods (Dropout, Data Augmentation, Early Stopping, BatchNorm)
- Goal: **avoid overfitting**



# From Regularization To Optimization

Optimization: Finding Good Solutions



## Optimization

- Even with good regularization, models won't learn unless we can find good parameters.
- *Optimization* = how we actually train models by minimizing the loss function.
- **Key challenge:** loss landscapes in ML are high-dimensional, noisy, and non-convex.
- We need smarter strategies than plain gradient descent.

# Optimization Techniques

1. Momentum
2. Nesterov Momentum (NAG – Nesterov Accelerated Gradient)
3. RMSProp (Root Mean Square Propagation)
4. Adam (Adaptive Moment Estimation)
5. AdamW (Adam with Weight Decay)

# Momentum Optimization

- Standard gradient descent can be slow (zig-zagging in narrow valleys).
- **Momentum** adds a "velocity" term that accumulates past gradients.
- This helps the optimizer keep moving in the right direction and damp oscillations.

$$v_t = \beta v_{t-1} + \eta \nabla L(w_t)$$

$$w_{t+1} = w_t - v_t$$

$v_t$ : Velocity vector at time  $t$  (accumulated gradient information).

$\beta$ : Momentum factor (typically a value between 0 and 1, often around 0.9).

$\eta$ : Learning rate.

$\nabla L(w_t)$ : Current gradient of the loss function with respect to weights  $w_t$ .

$w_t$ : Weights at time  $t$ .

# Nesterov Momentum

## NAG – Nesterov Accelerated Gradient

- A refinement of momentum.
- Instead of looking at the gradient at the current position, it “**looks ahead**” at the expected future position (where momentum will take it).
- This gives a correction term, leading to faster and more stable convergence.

$$v_t = \beta v_{t-1} + \eta \nabla L(w_t)$$

$$w_{t+1} = w_t - v_t$$



$$v_t = \beta v_{t-1} + \eta \nabla L(w_t - \beta v_{t-1})$$

$$w_{t+1} = w_t - v_t$$

# Nesterov Momentum

$v_t$ : Velocity vector at time  $t$  (accumulated gradient information).

$\beta$ : Momentum factor (typically a value between 0 and 1, often around 0.9).

$\eta$ : Learning rate.

$\nabla L(w_t)$ : Current gradient of the loss function with respect to weights  $w_t$ .

$w_t$ : Weights at time  $t$ .

Momentum

$$v_t = \beta v_{t-1} + \eta \nabla L(w_t)$$

$$w_{t+1} = w_t - v_t$$



Nesterov Momentum

$$v_t = \beta v_{t-1} + \eta \nabla L(w_t - \beta v_{t-1})$$

$$w_{t+1} = w_t - v_t$$

## Intuition

- **Momentum:** you move, then check the slope.
- **Nesterov:** you first “peek” ahead in the direction momentum would carry you, then adjust the step based on the slope there.

# RMSPProp

Gradient descent can get stuck if gradients are very different across dimensions (one steep, one flat).

RMSProp rescales gradients by dividing them with a running average of their past magnitudes.

**Update rule (simplified):**

$$E[g^2]_t = \beta E[g^2]_{t-1} + (1 - \beta)g_t^2$$

$$w_{t+1} = w_t - \eta \frac{g_t}{\sqrt{E[g^2]_t + \epsilon}}$$

- Each parameter gets its own adaptive learning rate.
- Helps damp steps in steep directions, accelerates in flat ones.
- Think of it as automatically adjusting the step size per dimension.

# Adam (Adaptive Moment Estimation)

Combines Momentum + RMSProp.

Keeps track of:

- First moment (mean of gradients)  $\rightarrow$  like momentum.
- Second moment (variance of gradients)  $\rightarrow$  like RMSProp.

Adds bias correction to avoid problems at the start

**Update rule (simplified):**

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$w_{t+1} = w_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

# Adam (Adaptive Moment Estimation)

Combines Momentum + RMSProp.

Keeps track of:

- First moment (mean of gradients) → like momentum.
- Second moment (variance of gradients) → like RMSProp.

Adds bias correction to avoid problems at the start

**Update rule (simplified):**

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$w_{t+1} = w_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

bias-corrected estimates

# Adam (Adaptive Moment Estimation)

1. **Calculate the gradients:** The gradient now includes a term for the L2 penalty.

$$g_t = \nabla_{w_{t-1}} \text{Loss}(w_{t-1}) + \boxed{\lambda w_{t-1}} \longrightarrow \text{L2 regularization (weight decay) implementation in Adam}$$

2. **Update biased first moment estimate:**

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

3. **Update biased second moment estimate:**

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

4. **Compute bias-corrected moments:**

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

5. **Update weights:** The weight update is applied using the bias-corrected moment estimates.

$$w_t = w_{t-1} - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

# Adam (Adaptive Moment Estimation)

Formulas:

1. First moment (mean):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

2. Second moment (variance):

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

3. Bias correction:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

4. Parameter update:

$$w_{t+1} = w_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} - \eta \lambda w_t$$

**Problem:**

this mixes the weight decay term into the moment calculations,

which can lead to incorrect interactions between adaptive learning rates and regularization.

# AdamW (Adam with Weight Decay)

- In standard Adam, weight decay (L2 regularization) interacts strangely with the adaptive updates.
- AdamW fixes this issue by decoupling weight decay from the adaptive updates.
- Instead of mixing it into the gradient, AdamW applies weight decay **directly** to the weights.

# AdamW (Adam with Weight Decay)

1. **Calculate the gradients:** The gradients are computed only with respect to the original loss function, not including a weight decay term.

$$g_t = \nabla_{w_{t-1}} \text{Loss}(w_{t-1})$$

2. **Update biased first moment estimate:**

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

3. **Update biased second moment estimate:**

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

4. **Compute bias-corrected moments:**

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

5. **Apply decoupled weight decay:** The weight decay is applied by scaling the current weight, before the adaptive update.

$$w_t = w_{t-1} (1 - \alpha \lambda)$$

6. **Update weights:** The standard Adam-style update is then applied.

$$w_t = w_t - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$



# Questions?

Email: [zara.hajihashemi@sjsu.edu](mailto:zara.hajihashemi@sjsu.edu)